



Qualcomm Technologies, Inc.

Qualcomm Linux Security Guide

80-70014-11 Rev. AR

August 6, 2024

Contents

1 Overview	6
2 Getting started	8
3 Features	10
3.1 Cryptography	11
3.2 Key management	12
3.3 Secure boot	13
3.4 Storage security	14
3.5 Storage encryption	14
3.6 SMC invoke	15
3.7 Access control	17
3.8 Secure peripheral image loading	18
3.9 SELinux	18
3.10 Qualcomm TEE	21
3.11 Qualcomm Hypervisor	22
3.12 Qualcomm WES	25
3.13 Security hardening	26
4 Architecture	27
5 APIs	32
6 Develop	34
7 Tools	35
8 Bring up	36
9 Configure	40
9.1 Enable secure boot	41
9.1.1 QFPROM fuses	43
9.1.2 Generate local (insecure) root key and certificates	48
9.1.3 Generate ECDSA root key and certificate	49
9.1.4 Generate RSA CA key pair and certificate	50
9.1.5 Generate SHA-384 hash for RSA and ECDSA	51

9.1.6 Sign images	51
9.1.7 Generate signed sec.elf image	51
9.1.8 Flash images	52
9.2 Enable SELinux	54
9.3 Enable UEFI secure boot	56
9.3.1 Generate key and certificate	57
9.3.2 Sign images and copy (.Auth) key/signed files to EFI partition	59
9.3.3 Enable UEFI secure boot from systemd-boot menu	61
9.3.4 Hash unsigned images and update db for image authentication	62
9.4 Sample OpenSSL configuration	63
9.5 Install or upgrade QCS5430 SoftSKU feature packs	70
10 Customize	74
11 Debug	75
12 Examples	78
A References	79

Figures

Figure 1-1: Qualcomm security foundations.....6

Figure 3-1: Hypervisor architecture..... 23

Figure 4-1: Qualcomm Linux security software components..... 28

Figure 9-1: SELinux process.....55

Figure 9-2: Systemd-boot menu options..... 61

Figure 9-3: UEFI secure boot enablement information from serial log.....61

Tables

Table 3-1: Objects – SMC invoke.....	15
Table 3-2: SMC invoke modules.....	17
Table 9-1: EFI partition layout.....	59
Table 11-1: Debug policy flags for dump collection.....	77

1 Overview

The Qualcomm® Linux product security offers a comprehensive security framework designed to protect devices from both software and hardware threats.

The secure foundations provided by Qualcomm ensure not only a high level of security and robustness but also superior performance, all while preserving energy efficiency. The major components of this framework are illustrated in the following figure.

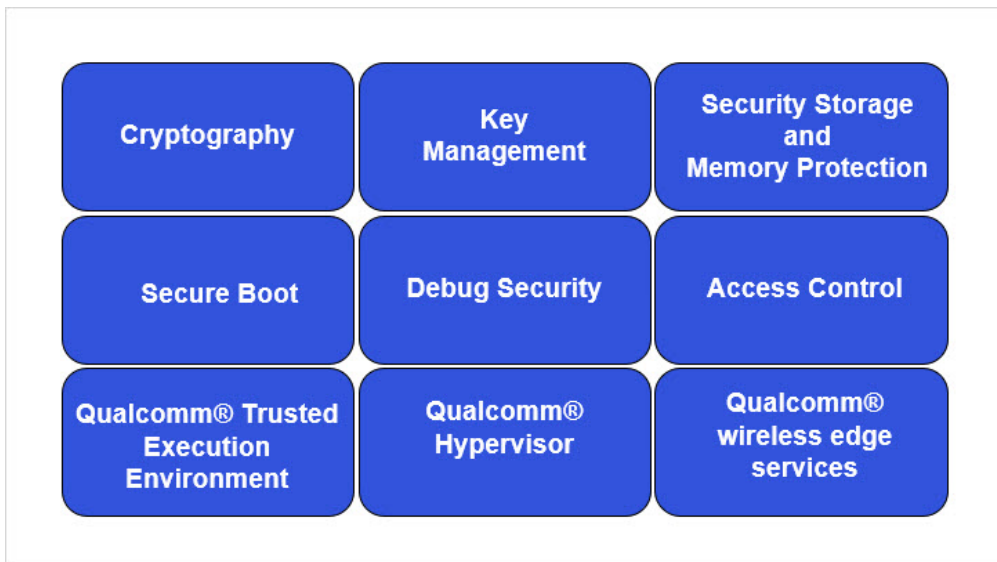
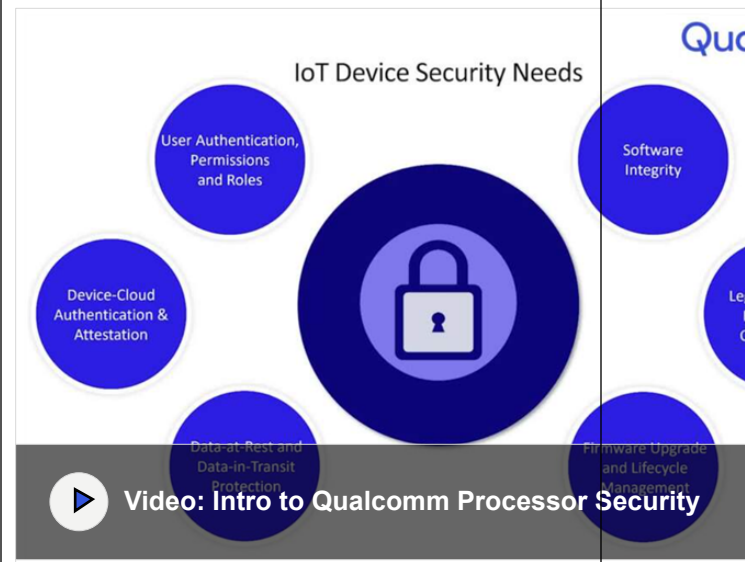


Figure 1-1 Qualcomm security foundations

Dive deep into the world of IoT security with our comprehensive guide focused on Qualcomm SoC's. Learn how to optimize and enhance the safety of your IoT products through advanced security solutions. This video covers everything from understanding potential threats like code modifications and key exposures to implementing foundational security requirements. Whether you're dealing with biometric authentication or cloud connectivity, discover how Qualcomm's security features can safeguard your devices against evolving cyber threats and meet stringent regulatory standards. Join us to fortify your IoT devices now!



The following is a list of the functionalities of each block within the Qualcomm secure foundations:

- The [Cryptography](#) goal is to ensure more secure and faster encryption/decryption operations using crypto engines and hardware keys.
- The [Key management](#) process uses a one-time programmable (OTP) eFuse memory known as the Qualcomm fuse programmable read only memory (QFPROM) to store generic information and configurations.
- The [Secure boot](#) feature offers image authentication and tamper-resistant RoT in electronic fuses (eFuse).
- The debug security feature allows debugging in a secure environment. It is set by an eFuse and restricts the invasive (INV) and non-invasive (NINV) debug capabilities in commercial products and reverse engineering. For more information, see [Learn the architecture - Before debugging on Armv8-A](#).
- The [Storage security](#) feature offers encrypted storage for digital rights management (DRM) keys and certificates. These keys are securely provisioned. These keys are provisioned securely. The replay protected memory block (RPMB) is a separate partition in the universal flash storage (UFS) device designed for secure data storage.
- The [Access control](#) feature prevents unauthorized code execution using the access-controlled memory and peripherals.
- The [Qualcomm TEE](#) isolates the secure and non-secure software operations. It is built on the Arm® TrustZone® (TZ) architecture and has a rigorously reviewed small code base.
- The [Qualcomm Hypervisor](#) allows multiple operating system environments to run simultaneously and independently.
- The [Qualcomm WES](#) is a cloud-based service that ensures trust throughout the device lifecycle using hardware root-of-trust (RoT). It provides services like hardware-based attestation, zero-touch device provisioning, and chipset feature management.

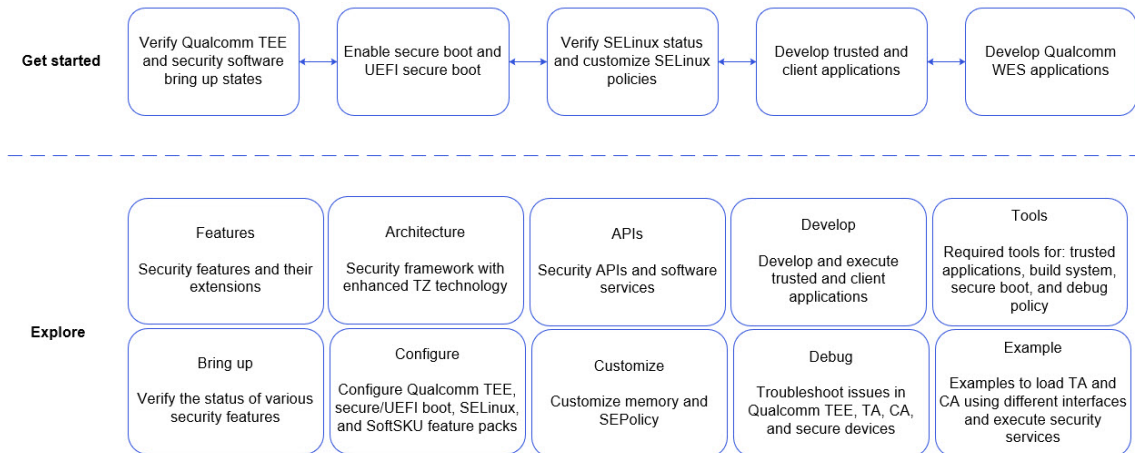
NOTE The Qualcomm Linux platform allows you to develop applications for QCS6490 and QCS5430.

2 Getting started

This information explains how to get started with developing your software using the Qualcomm Linux platform.

Before you begin, set up your infrastructure as described in the [Qualcomm Linux Build Guide](#). This guide also provides information about the common build workflows.

After your infrastructure is set up, you can now get start by either verifying the various security features or explore the other sections as shown in the workflow.



You must enable secure shell (SSH) in the Permissive mode to securely access your host device. For instructions on how to enable SSH and connect to the device, see [Qualcomm Linux Build Guide → How to → How to SSH](#).

Verify Qualcomm TEE and security software bring up state

When you have access to the development kit, and have built and loaded the software on the hardware, your infrastructure setup is ready. You can verify the availability of TZ (secure) execution environment.

For instructions, see [Bring up](#).

Enable secure boot and UEFI secure boot

One of the important aspects of achieving hardware security and IP protection is enabling secure boot for the hardware.

For more information and instructions, see:

- [Secure boot](#) and [Enable secure boot](#)
- [UEFI secure boot](#) and [Enable UEFI secure boot](#)

Verify SELinux status and customize SELinux policies

It is important to verify the SELinux status for the device software to ensure there is no non-privileged access to the critical software modules or drivers.

For more information and instructions, see:

- [SELinux](#) and [Enable SELinux](#).
- [SEPolicy](#)

Develop trusted and client applications

To execute the security critical applications in the secure world (TZ), you must develop your own trusted applications (TA). You must have access to the Qualcomm TEE software development kit (SDK) to develop corresponding client applications that launch the TA.

For more information, see [Develop](#).

Develop Qualcomm WES applications

If you have a requirement to develop the applications that provide hardware-based attestation, zero-touch device provisioning, and the chipset feature management, see:

- [Qualcomm WES](#)
- [Develop](#)

NOTE In this guide, the term OEM refers to both the developer and the original equipment manufacturer.

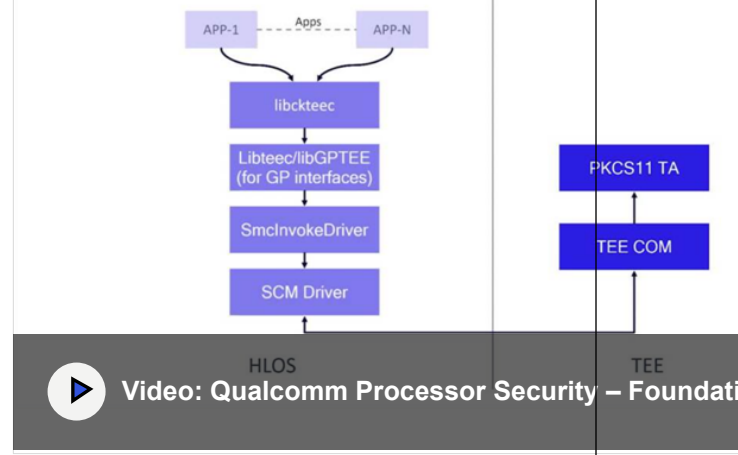
3 Features

Qualcomm TEE enhances the security features and their extensions. It offers interfaces that allow the extension of the security feature set through TA. Certain features are integrated into the hardware-supported TZ architecture, providing a system security configuration. These features can be further customized to meet specific requirements.

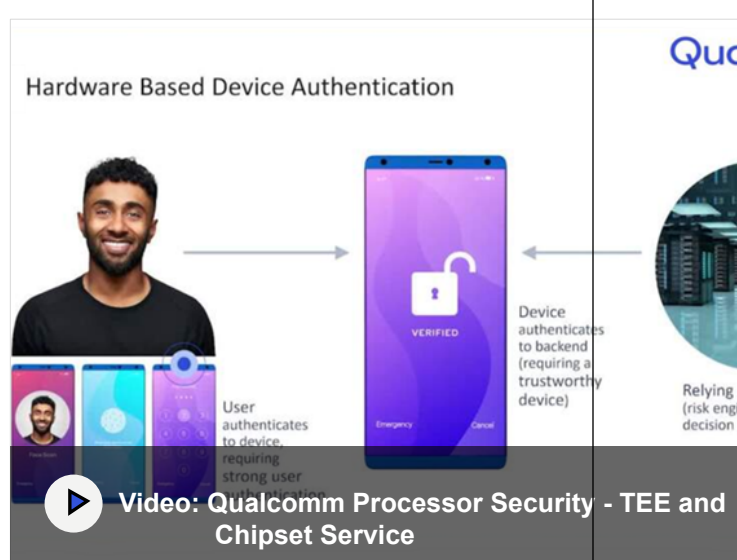
For information on Qualcomm TEE and secure components, see [Qualcomm TEE](#) and [Architecture](#).

Unlock the full potential of Secure Boot technology on Qualcomm devices in this comprehensive tutorial. From generating cryptographic keys to programming hardware fuses and managing secure boot status, this video covers every step in detail. Ideal for developers aiming to enhance device security through authenticated boot processes. Learn how to use Qualcomm's tools effectively to ensure your device boots securely every time.

Key Management



Learn how to use Qualcomm's Type 1 Hypervisor and Trusted Execution Environment in this comprehensive tutorial. Dive deep into platform virtualization, secure communication, and the extensive security features offered by Qualcomm Technologies. Learn how to develop secure applications using Qualcomm's tools and APIs, and understand the critical security use cases and compliance standards that Qualcomm supports. Perfect for developers aiming to enhance device security and functionality.



NOTE The information related to features in this guide is descriptive and intended for your understanding. For any configurations or customizations, see [Configure](#) and [Customize](#).

3.1 Cryptography

Qualcomm Linux security offering includes support for both hardware and software-based cryptography.

The key capabilities include:

- A register and bus access manager with direct memory-based access.
- Interfaces to the cryptographic hardware.
- - The Linux kernel crypto driver (qcrypto) provides access to the hardware cryptography independent of TAs.
 - The Qualcomm TEE provides the hardware and software crypto application programming interfaces (APIs) to the TAs.

Qualcomm TEE supports the following cryptographic algorithms:

Algorithm	Hardware	Software
Hash	SHA-1/SHA-256	▪ SHA-1/SHA-224/SHA-256/SHA-384/SHA-512 ▪ SM3
Symmetric cipher	▪ AES-128/AES-256 CBC, ECB, CTR, CCM, GCM, ▪ Triple-TDES CBC/ECB	▪ AES-128/AES-192/AES-256 CBC, ECB, CTR, CCM, XTS, CFB, OFB, CTS ▪ Triple-TDES CBC/ECB ▪ PBKDF2 ▪ SM4
MAC	AES-CMAC,	Hash-based message authentication (HMAC)
RNG	HRNG	—

Algorithm	Hardware	Software
HMAC	HMAC-SHA-1/SHA-256	HMAC-SHA-1/SHA-224/SHA-256/ SHA-384/SHA-512
Asymmetric cipher	—	▪ RSA with 1024/2048/3072 modulus ▪ ECDSA with P224, P256, P384, P521 ▪ ECDH ▪ SM2

Inline crypto engine

The inline crypto engine (ICE) is designed for a high throughput cryptographic encryption of the storage data.

The features of ICE include:

- Support for AES 128/AES 256 ECB/XTS
- Support for multiple crypto streams to meet high throughput
- Support for multiple AES cores per crypto stream
- Provision of 32 software configurable keys
- Capability to enable symmetric and asymmetric operations

3.2 Key management

The Qualcomm security solution supports the public-key cryptography standards (PKCS#11) by implementing the PKCS#11 APIs. This allows applications to use keys and certificates in a platform-independent manner.

PKCS#11 is implemented as a global platform trusted application (GP TA) within Qualcomm TEE. There is also a corresponding rich execution environment (REE) implementation for these applications.

For more information, see the following documents:

Document	Link
<i>PKCS #11 Cryptographic Token Interface Base Specification Version 2.40</i>	http://docs.oasis-open.org/pkcs11/pkcs11-base/v2.40/os/pkcs11-base-v2.40-os.html
<i>PKCS #11 Cryptographic Token Interface Usage Guide Version 2.40</i>	https://docs.oasis-open.org/pkcs11/pkcs11-ug/v2.40/cnd01/pkcs11-ug-v2.40-cnd01.html

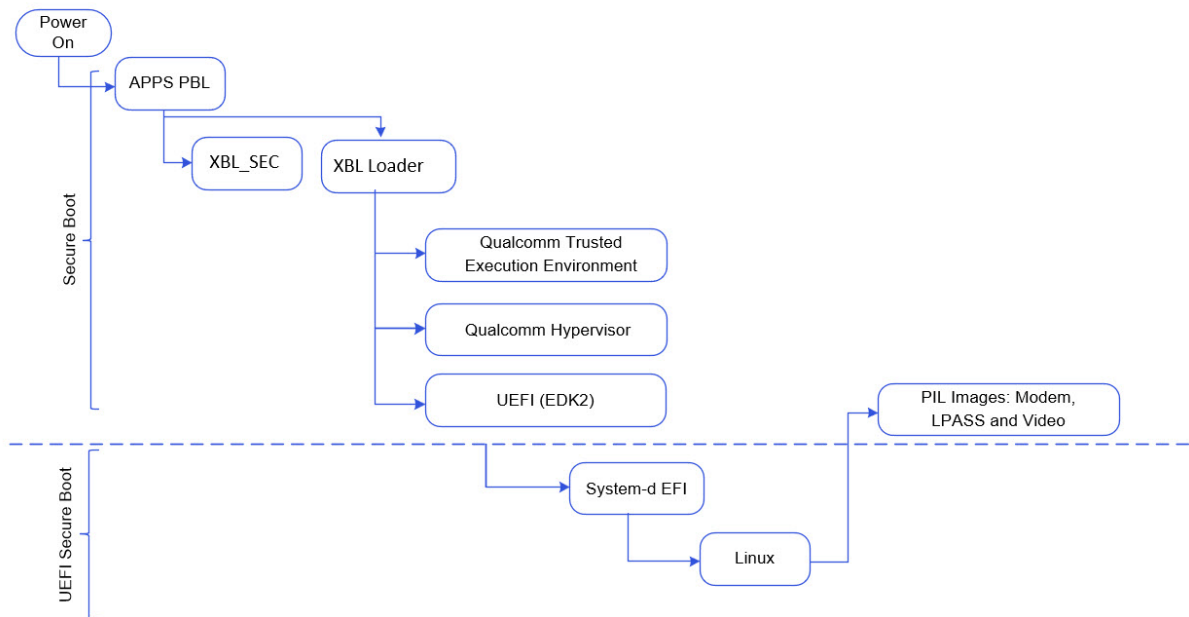
Limitations

The following known limitations are not supported:

- Random number generator functionality
- P-192 in CKM_ECDSA
- RSA PKCS key generation and signing in CKM_RSA_PKCS mode
- EDDSA key generation and signing

3.3 Secure boot

Secure boot is the boot-up sequence that establishes a trusted platform for the entire software stack. See the workflow to understand the secure boot and UEFI secure boot process.



This process ensures that only authorized software is executed. It uses cryptographic authentication to initiate an immutable sequence that validates the origin of the code. This process:

- Confirms the authenticity of all software images (non-Linux images) signed by Qualcomm and the OEMs/developers. The device carries out this process.
- Prevents any unauthorized or maliciously modified software from running on the device.

For information on enabling secure boot, see [Enable secure boot](#).

The secure boot feature authenticates non-Linux images. For authentication The Linux images can be authenticated using UEFI secure boot.

UEFI secure boot

UEFI secure boot is a feature of the unified extensible firmware interface (UEFI) specification that defines an interface between an operating system and platform firmware.

For more information on UEFI specification, see <https://uefi.org/specs/UEFI/2.10/>.

The features of UEFI secure boot include:

- Ensuring that the code executed by the device's UEFI firmware is secure and trusted before the operating system begins boot up.
- Defining how the UEFI authenticates images such as the Linux image, the OS loader (uki.efi), systemd boot (bootaa64.efi), and the device tree blob (DTB) image files.
- Ensuring that the images are only loaded when signed by valid and authorized users. This process also ensures the platform's security and integrity over systems running on UEFI-based firmware.

The UEFI secure boot allows the platform owner to:

- Verify the integrity and security of a UEFI loaded image, ensuring it is loaded in an approved manner.
- Manage the platform security policy as defined by the UEFI secure boot authenticated variables, which include:
 - Platform key (PK)
 - Key exchange keys (KEK)
 - Allowed database (DB)
 - Forbidden database (DBX) is not supported in this release.

For more information, see [Enable UEFI secure boot](#).

3.4 Storage security

The secure file system (SFS) is used to store sensitive data, such as keys and biometric data.

SFS

SFS provides confidentiality, integrity, and anti-rollback support to TA and securely stores sensitive data. Any file created or stored under SFS is covered by anti-rollback protection. The SFS feature:

- Uses an encryption key for each TA to ensure the confidentiality of the files.
- Uses an HMAC key for each TA to verify the integrity of the files.

Both the encryption and HMAC keys are derived using a device unique key, which depends on the secure boot state of the device. The SFS anti-rollback protection is enabled by default.

When the devices are secure boot enabled, the SFS uses unique hardware keys for file data encryption and decryption to ensure they are secure from each other.

For information on enabling secure boot, see [Enable secure boot](#).

RPMB

RPMB is a physical partition on the UFS/eMMC flash. This partition is used to store sensitive information and is only accessible from Qualcomm TEE.

To read from and write to the RPMB partition, RPMB key provision is required. This is a one-time process that cannot be overwritten or erased when completed.

To provision and enable RPMB, see [RPMB-based SFS antirollback protection](#).

Every access to the RPMB is authenticated, allowing the host to store data in an authenticated and replay-protected manner.

3.5 Storage encryption

Storage encryption enhances security by supporting the transparent encryption of files and directories.

The Qualcomm platform supports storage encryption with the [Inline crypto engine](#) and a hardware-wrapped key. It provides better efficiency and enhanced key protection.

The storage encryption feature allows:

- Provision of standard keys ranging from 32 bytes to 64 bytes for content encryption.
- Encryption of filenames and file content with separate keys.
- Generation of 32-byte key identifiers.

The Qualcomm platform UFS storage driver is added to support the `fsencrypt` API.

NOTE The eMMC support will be made available in a future release.

For more information on the `fsencrypt` API, see the kernel documentation at `Documentation/filesystems/fsencrypt.rst`.

For related kernel documentation, see the following files:

- Inline encryption: `Documentation/block/inline-encryption.rst`
- IOCTL support for storage encryption: `Documentation/userspace-api/ioctl/ioctl-number.rst`

You can invoke the storage encryption functionality using the open-source `fsencryptctl` tool, which can be found at <https://github.com/google/fsencryptctl>.

3.6 SMC invoke

The secure monitor call (SMC) invoke is used to expose the services and interfaces implemented in Qualcomm TEE to Linux clients. It provides identification information about the requesting processes to Qualcomm TEE.

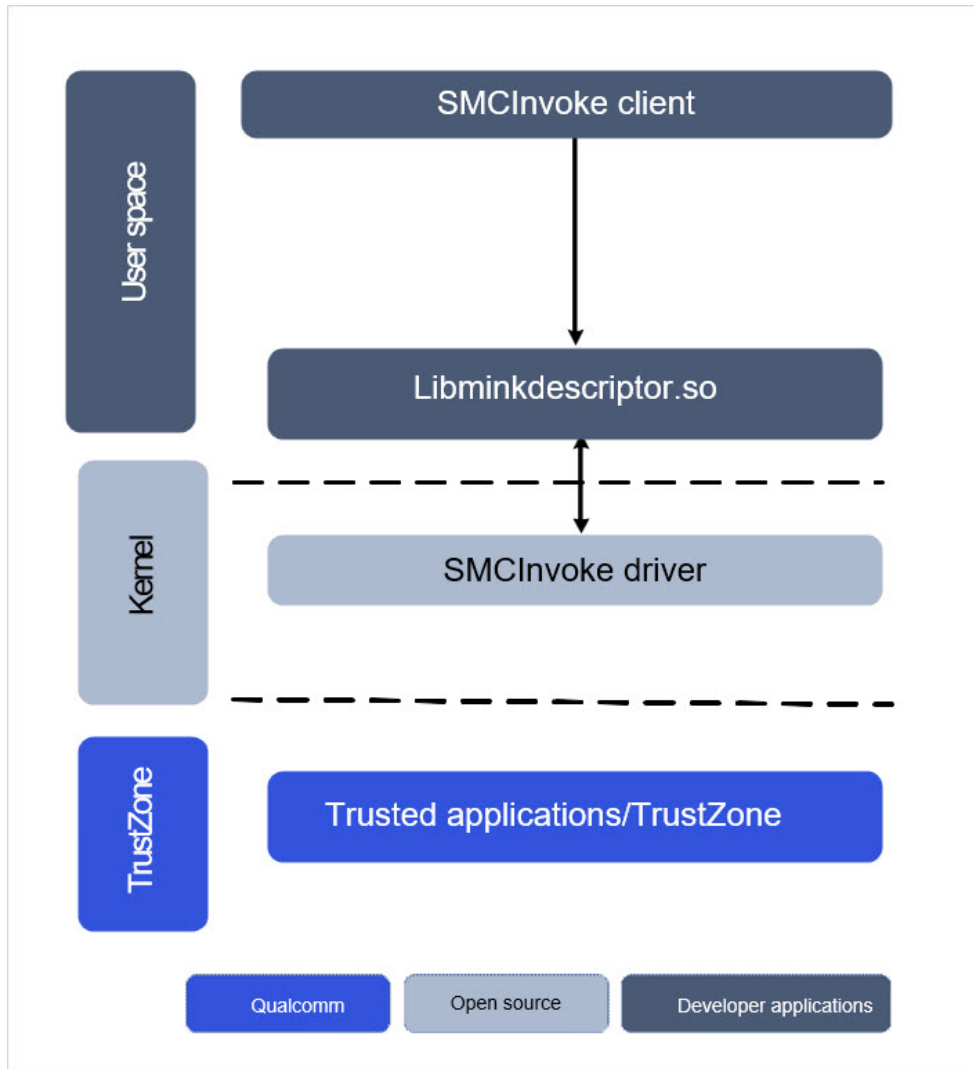
The SMC invoke is an object-oriented, capability-based framework that enables communication between Linux clients, TZ, and the TZ applications, which use mini-kernel (MINK) invoke objects.

During the SMC invoke process, the following objects are involved:

Table 3-1 Objects – SMC invoke

Object	Owner environment	Description
Remote objects	TZ execution environment	Linux obtains references to remote objects through the invoke operations. For example, an AppLoader, which is a remote object.
Callback objects	Linux execution environment	Linux sends references of the required callback objects through the invoke operations to TZ, which may invoke these objects as needed.
Memory objects	Linux execution environment	These memory objects are useful for efficiently sharing large buffers between TZ and Linux.

The figure shows the SMC invoke architecture:



The SMC invoke has the following components:

- Linux kernel:

The SMC invoke driver in the kernel is responsible for transporting user space requests between the TZ and the Linux user space clients. It exposes the following input/output controls (IOCTL).

- SMCINVOKE_IOCTL_INVOKE_REQ
- SMCINVOKE_IOCTL_ACCEPT_REQ
- SMCINVOKE_IOCTL_SERVER_REQ
- SMCINVOKE_IOCTL_ACK_LOCAL_OBJ

- Linux user space:

The SMC invoke in the user space includes `libminkdescriptor.so` and the SSGTSD daemon.

Table 3-2 SMC invoke modules

Module	Description
SMCInvoke client	This client communicates with TAs in Qualcomm TEE through the SMC invoke driver.
TA client	This client communicates with TZ applications through the SMC invoke driver.
Libminkdescriptor	- The module creates a bridge between user space client and the kernel. - A client using SMC invoke must use <code>libminkdescriptor</code> to communicate with the kernel. - The <code>libminkdescriptor.so</code> is available at <code>/usr/lib/libminkdescriptor.so</code>
SMCInvoke driver	- The driver in the Linux kernel communicates with application clients and TAs in TZ. - The SMC invoke driver source code is available at <code>/kernel/drivers/soc/qcom/smcinvoke/smcinvoke.c</code>.

3.7 Access control

The access control trust model ensures ongoing security and manages access control configurations among assets, interfaces, SoC components, and images.

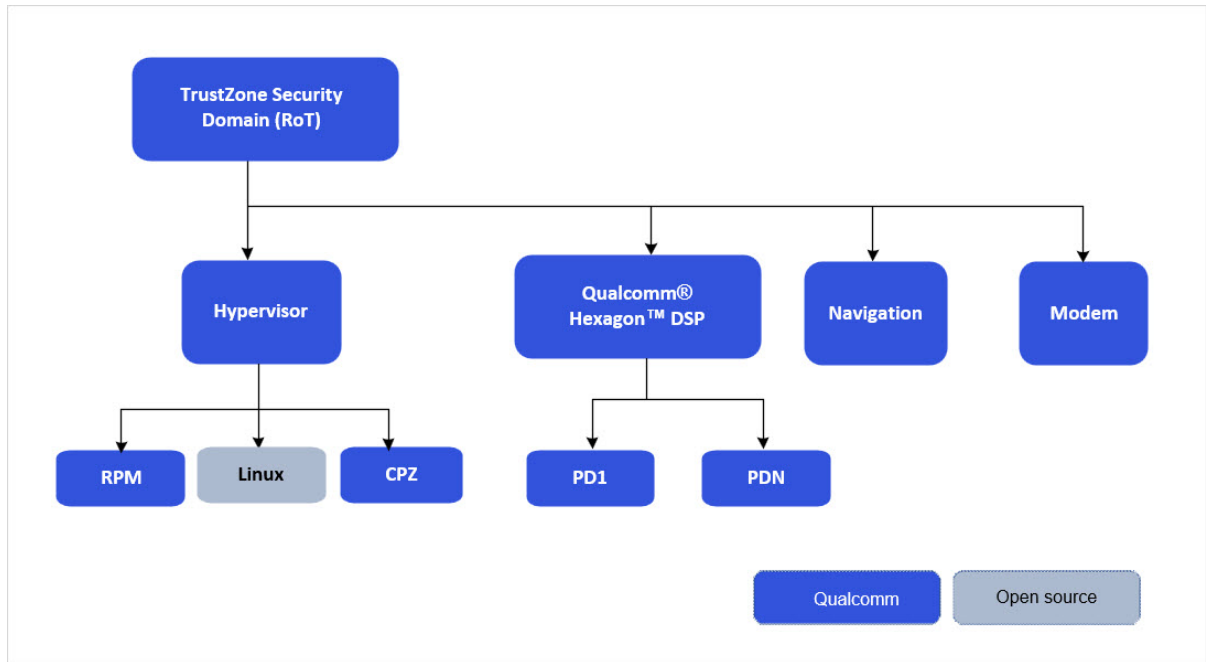
- The Qualcomm access control uses an external protection unit (xPU) to control access from the slave side for registers, fixed address, and dynamic memory regions.
- The system memory management unit (MMU) controls access from the master side for content protection and subsystem memory sharing use cases.

Access control domains

There are two levels of control:

1. The TZ manages the TZ domains and controls access from the slave side using xPUs.
 - An xPU is a combination of multiple security blocks known as protection units. It allows conditional access based on a set of programmable access control registers.
 - If access is denied, the xPU generates an error output signal and, if necessary, an interrupt request signal.
2. The hypervisor (EL2) manages the non-secure domains, including the content protection zone (CPZ), which is designed to prevent access to premium videos.
 - It configures access control from the master side through second-stage page tables via the MMU.
 - It manages stage-2 mappings to protect assets across all masters (Linux, display, GPU, and video). The Linux kernel manages stage-1 mappings to protect the assets in applications that run at the user privilege level.

The figure illustrates the access control domain.



3.8 Secure peripheral image loading

The secure peripheral image loading (PIL) of a TZ authenticates different images and configures the xPUs for all subsystems such as audio, camera, and video.

For more information about xPU, see [Access control](#).

3.9 SELinux

SELinux is a security enhancement for Linux, providing greater control over system access. It implements mandatory access control (MAC) in the Linux kernel using the Linux security modules (LSM) framework.

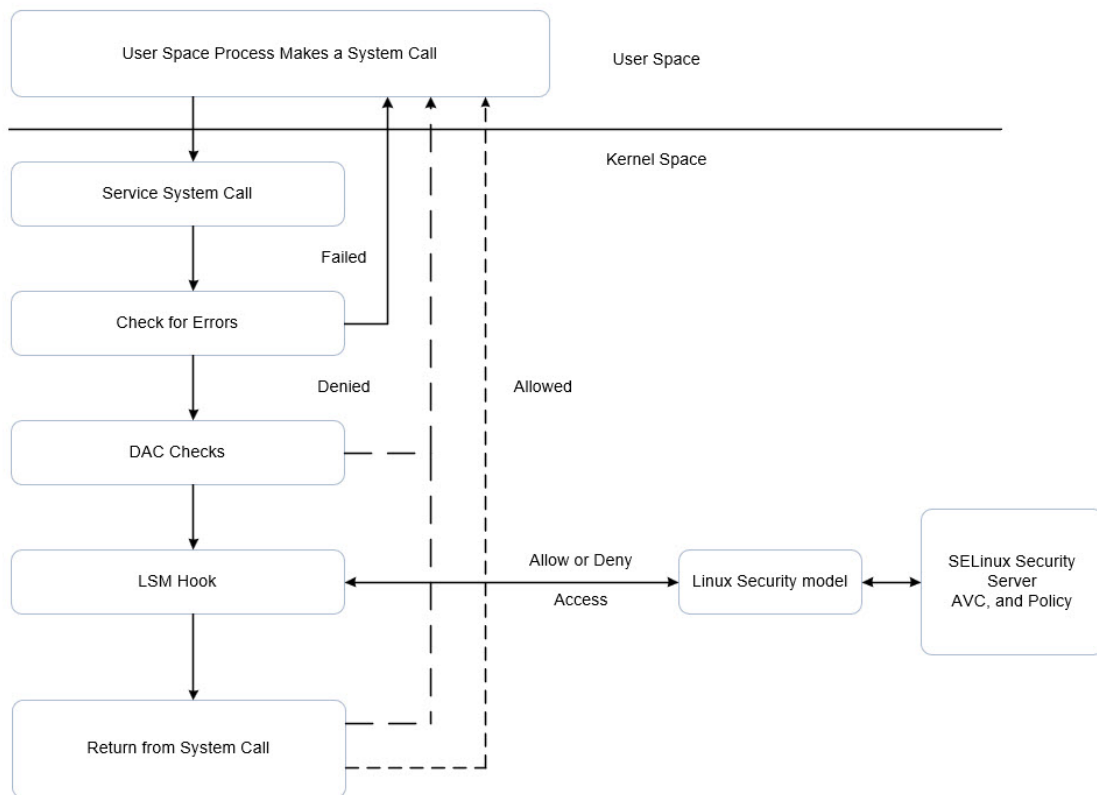
The LSM framework includes the following controls:

- Discretionary access control (DAC)
 - This is the standard form of Linux access control. The data owner has full control over the resource (such as data and files) and can update or modify the data.
 - Some applications (with root privilege) can override this control. Some resources, like the socket, are unchecked.
 - Access controls are provided with limited granularity, based on user and group identity. For example, File mode `-rwxr-xr-x-`

- **MAC**

- A system-wide security policy determines the Linux access controls for all processes, objects, and operations.
 - The policy can confine flawed and malicious applications or incautious users. It can even prevent users such as *root* or UID 0 from escalating privilege.
 - MAC provides finer granularity to access controls, allowing access to many more resources other than files with more specific controls. For example, unlink, append only, or move a file.

The figure shows the steps in the decision-making chain for DAC and MAC:



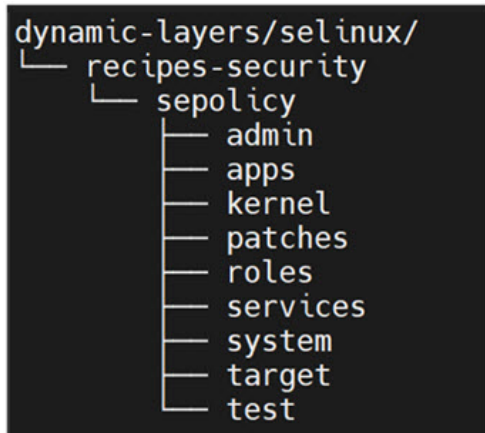
SELinux supports three modes:

- **Enforcing mode (Default):** In this mode, the SEPolicy is enforced on the system. If the SEPolicy rules are not met when software is executed, access is prevented. The kernel logs an attempted access violation as an access vector cache (AVC) denial message to *dmesg* and *journalctl*.
- **Permissive mode:** In this mode, the SEPolicy is not enforced on the system. All denials are logged into *dmesg* and *journalctl* logs, but access of a process or software is not disallowed.
- **Disable mode:** In this mode, the SEPolicy is neither enforced nor logged.

For information on enabling SELinux, see [Enable SELinux](#). To configure the SELinux modes, see [SELinux configuration](#).

SELinux layers

The SELinux dynamic layer includes the necessary SELinux-specific code changes that enable and bring up the device when SELinux is enabled with a MONOLITHIC design.



This layer is built on top of the meta-selinux layer, which is part of the Yocto upstream. It includes:

- The recipe-kernel, which contains the recipe to enable the required kernel configuration flags for SELinux.
- The recipe-security, which includes:
 - Policies for booting the device to the shell.
 - Policies for Qualcomm services and test applications.
 - Policies of upstream services in the form of patches.
 - A recipe to handle compilation of Qualcomm policies

For more information, see <https://www.redhat.com/en/topics/linux/what-is-selinux>.

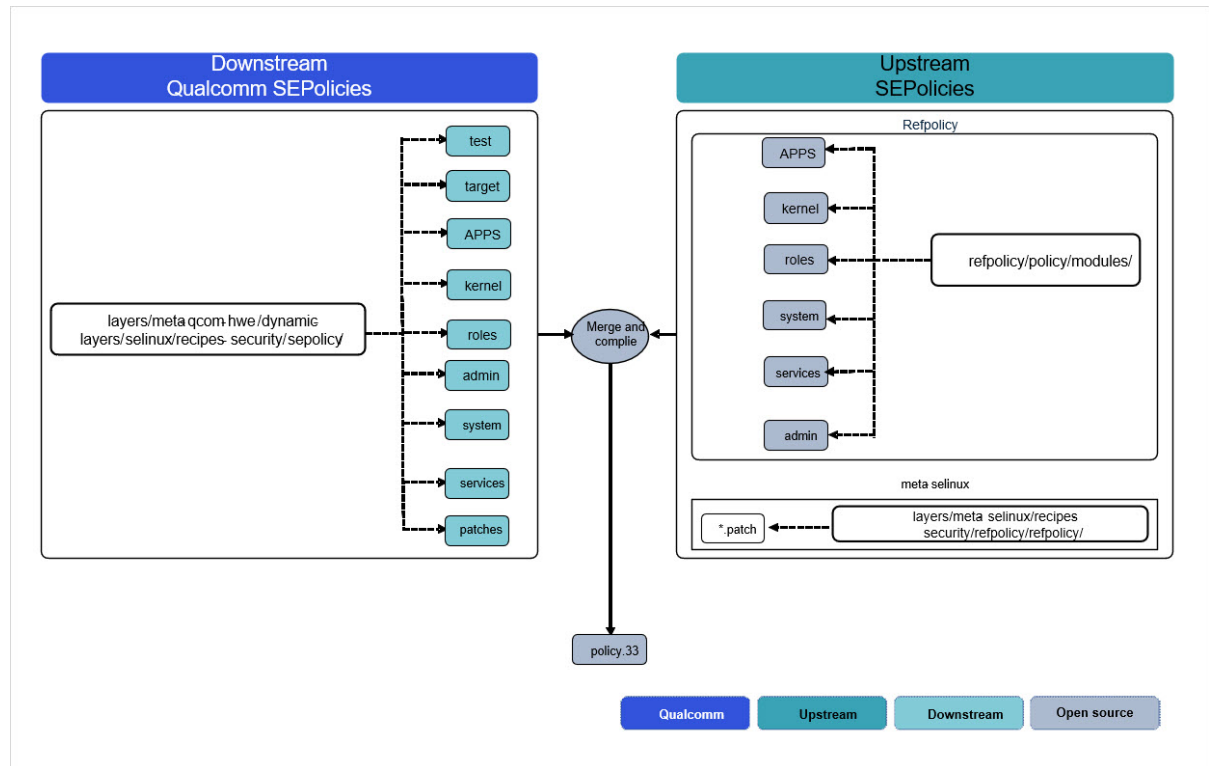
SEPolicy

SEPolicy is a directory that contains the core SELinux policy configuration.

The upstream SEPolicy is defined in `sepolicy`, which is downloaded during the build and amended with Qualcomm SEPolicy. The Qualcomm SEPolicy is at `layers/meta-qcom-hwe/dynamic-layers/selinux/recipes-security/sepolicy/`.

The upstream folder structure is adhered to, with `apps` / `kernel` / `system` / `services` added to the respective folder.

The figure shows the high-level folder structure that is merged and compiled to generate `policy.33`, which is the SEPolicy binary used or loaded during boot.



For more information, see [SEPolicy customization](#).

3.10 Qualcomm TEE

Qualcomm TEE is the software that operates within the Arm TZ environment on the Qualcomm device.

TZ is a hardware-based security architecture enabled through a Secure mode of the Arm processor. It establishes two execution environments with system-wide hardware-enforced isolation. For more information, see <https://developer.arm.com/documentation/102418/0101/What-is-TrustZone->.

Qualcomm offers a 64-bit Arm 8.x processor system with hardware virtualization to run TZ.

In the TZ architecture, there are two security states:

- Secure
- Non-secure

At the EL0, EL1, and EL2 [exception levels](#), the processor can be in either the secure state or the non-secure state, while EL3 is always in the secure state.

The operating system runs in non-secure EL1. The transition from Non-secure to Secure mode is facilitated through a Secure Monitor mode.

Qualcomm TEE provides the following features:

- Operation from hardware-protected memory
- Support for power collapse of security blocks such as the crypto engine, PRNG, inline crypto engine, and external protection units (xPU).

- Support for a secure peripheral image loader (PIL)
- Support for subsystem restart
- Provision of content protection
- Support for running TA
- Support for fuse management

Trusted applications

TA offer services within a secure environment for Linux clients that are not secure. QualcommTEE extends the following services to TA:

- Support for TAs to operate in the secure world at EL0
- Sand-boxing environment for TA
- Position-independent loading of TA
- Message passing between different TAs

TA operates from the memory that is protected by the hardware. However, the applications that require more memory have the option to load and run from double data rate (DDR) memory instead. By default, an application is set to run from hardware-protected memory.

3.11 Qualcomm Hypervisor

The Qualcomm Hypervisor provides a modern virtualization framework that allows multiple operating systems to run independently and concurrently, delivering high performance.

The Qualcomm Type 1 Hypervisor facilitates the hosting of multiple trusted execution environments for secure use cases. The figure shows the architecture of the Qualcomm Hypervisor software stack, its

components, and virtual machines (VM). This figure includes an example of one guest VM using the Linux kernel.

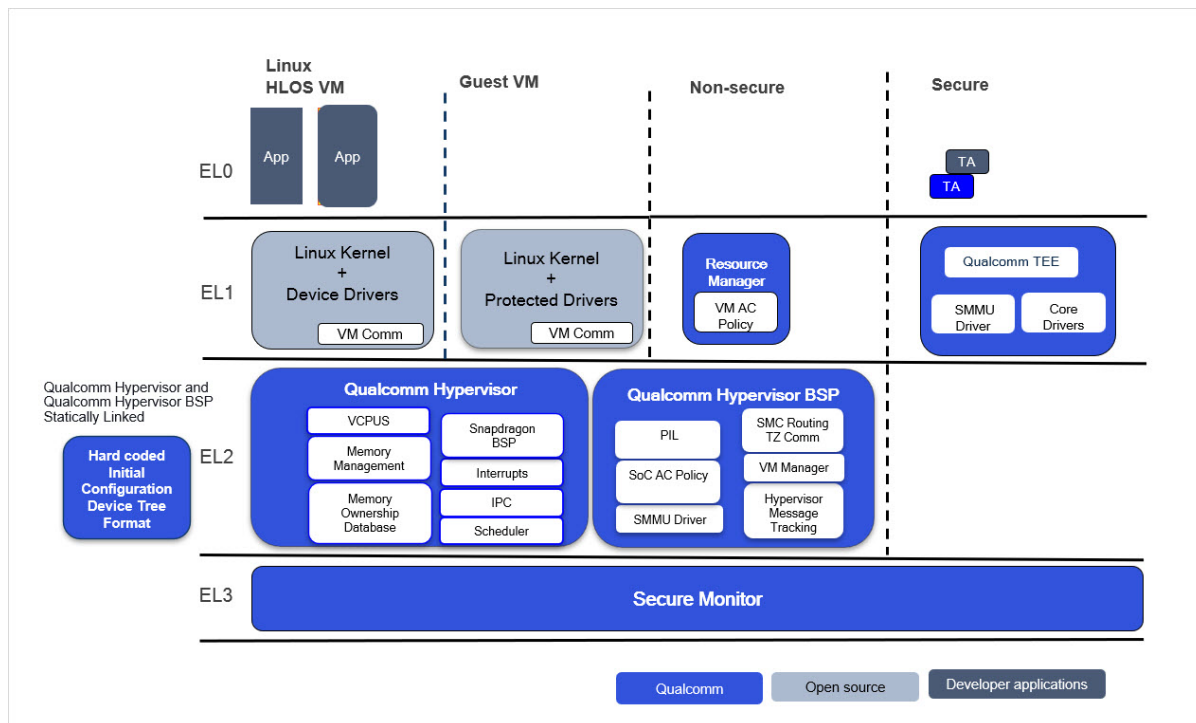


Figure 3-1 Hypervisor architecture

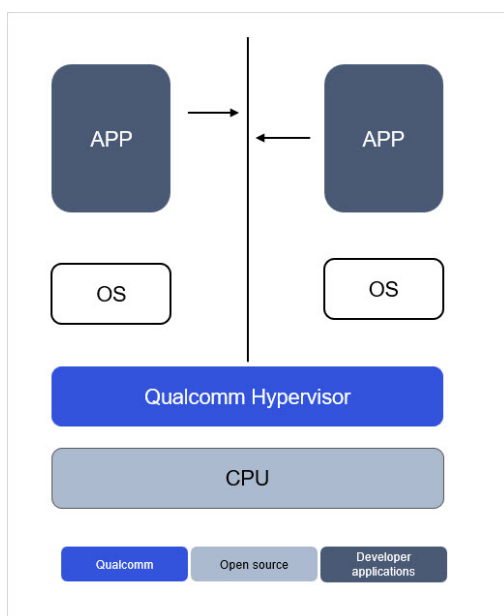
The Qualcomm Type 1 Hypervisor, also known as a native hypervisor, offers the following features:

- Provides enhanced security compared to Type 2 (hosted) hypervisors.
- Supports multi-core, real-time operations that are independent of Linux.
- Operates distinctly, independently, and with higher privileges than Linux.
- Has a minimal impact on performance.

The architecture of the Type 1 Hypervisor includes Qualcomm Hypervisor access control, which supports:

- Multiple operating systems
- Smaller attack surface

The figure illustrates the Type 1 Hypervisor.



Virtualization

VMs and virtual CPUs provide flexibility and abstraction. The Qualcomm Hypervisor offers the following virtualization features:

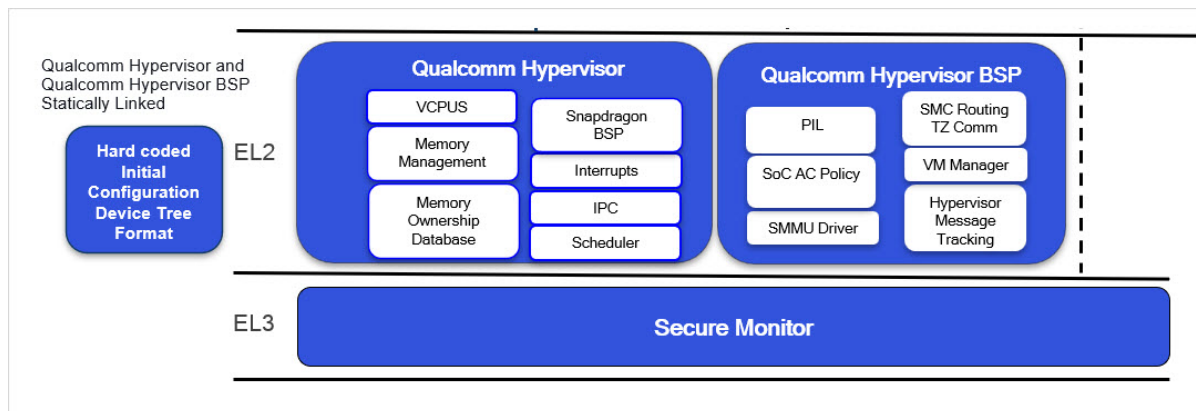
Feature	Description
Memory management	- The Qualcomm Hypervisor manages stage-2 page tables and translates the intermediate physical address (IPA) to the physical address (PA). - It also isolates memory between VMs.
Interrupt virtualization	It virtualizes and maps interrupts to the correct and independent VM.
Scheduling	The Virtualization CPU (VCPU) scheduler prevents VM starvation during failure.
Interprocess communication (IPC)	This includes shared memory, message passing (IPC) APIs, and virtual interrupts.
Power management	It supports multiple VMs and adopts the lowest common denominator power state.

Qualcomm Hypervisor BSP

The Qualcomm Hypervisor supports the following features within the board support package (BSP):

- PIL
- Access to SoC AC policy
- System MMU driver
- Pathway support for SMC TZ communication

The figure shows all the features from Qualcomm Hypervisor and BSP, which together form a comprehensive hypervisor solution:



Resource manager

The exception level 1 (EL1) resource manager performs the following functions:

- Creates and manages VMs
- Handles and assigns the VM AC policy
- Manages the I/O pass-through, which includes:
 - I/O space mapping and isolation
 - Interrupt routing configuration

For more information, see [Qualcomm Linux Kernel Guide → Features → Virtualization](#).

3.12 Qualcomm WES

Qualcomm WES is a suite of trusted services, rooted in hardware, that securely connects and manages devices.

It offers the following services:

- Feature licensing enables device features identified by a feature ID and an alias feature name. These features are enabled by installing the corresponding feature licenses or certificates.
- Device attestation provides cryptographically signed and encrypted data items that describe the security state of the device and its software. These data items are useful for risk engines and other similar applications.
- Secure provisioning enables the generation and use of cryptographic keys that are secured using unique device keys. These keys are used to:
 - Securely provision data to the device post-sale and over-the-air (OTA).
 - Sign data on the device.

For more information on upgrading feature pack, see [Install or upgrade QCS5430 SoftSKU feature packs](#).

3.13 Security hardening

Security hardening is a process that minimizes the risk of system attacks by making it more difficult an attacker to exploit the system vulnerabilities.

Kernel security hardening aligns with upstream kernel guidelines. Key kernel flags like KASLR, hardened user copy, stack protector, permissions (RWX) are enabled.

User space hardening

The `security_flags.inc` file, part of the [Yocto Project](#) is used to enable security compiler and linker flags for a build.

To extend this to Qualcomm modules, add the following command to `qcom-security_flags.inc` (file path: [layers/meta-qcom-distro/conf/distro/include/qcom-security_flags.inc](#)):

```
require conf/distro/include/security_flags.inc
```

Adding these flags may result in warnings or errors that can disrupt a build. However, Yocto provides a way to disable certain compiler flags for problematic packages. Modern compilers such as GCC and clang offer a wide range of compiler flags that can make it more difficult for an attacker to exploit certain types of vulnerabilities.

The following are the example flags with GCC:

- The `Wformat` flag adds compile-time checks to detect issues related to the format of string arguments in common library functions such as `printf`, `scanf`, and `strftime`.
- The `D_FORTIFY_SOURCE` flag adds compile and runtime checks to detect buffer overflows in memory and string functions
- The `Fstack-protector` flag adds runtime checks to detect buffer overflows and stack smashing.
- The `Fpie` flag enables position-independent code, which allows for loading the binary at randomized locations, thus making certain types of attacks (like return-oriented programming) more difficult.
- The `Wl,-z,relro,-z,now` flag makes it harder to abuse a binary global offset table.

If there are warnings and errors, customizing these flags for some modules can break a build. The binaries in a file system can be verified if the compiler exploit mitigation features are applied using a tool like Checksec.

For more information, see [The Yocto Project® dev documentation - Making images more secure](#).

4 Architecture

Armv8 has enhanced the TZ technology to incorporate security into its architecture. This technology offers a security framework that allows a device to handle security threats at both software and hardware levels.

NOTE The architectural information is provided for comprehension purposes only. You do not need to perform any configuration or customization.

The hardware solution from Arm allows the design and implementation of software, applications, or services that run in a secure environment. This environment is a separate execution unit that ensures hardware isolation from non-secure execution environments. For more information, see [Qualcomm TEE](#).

Qualcomm Linux operates on a 64-bit Arm 8.x architecture. In this setup, Arm cores have two execution modes:

- Non-secure mode: Linux operates in this mode of the Arm core
- Secure mode: Qualcomm TEE and TA operate in the Secure mode of the Arm core

The secure mode of the Arm core forms the essence of the TZ technology. It provides a hardware-based security environment for a secure OS and separates the secure world from the non-secure world.

The Qualcomm Linux security software architecture is depicted in the following figure. It shows the distribution of components or modules across both the rich (user space and kernel space) and trusted execution environments.

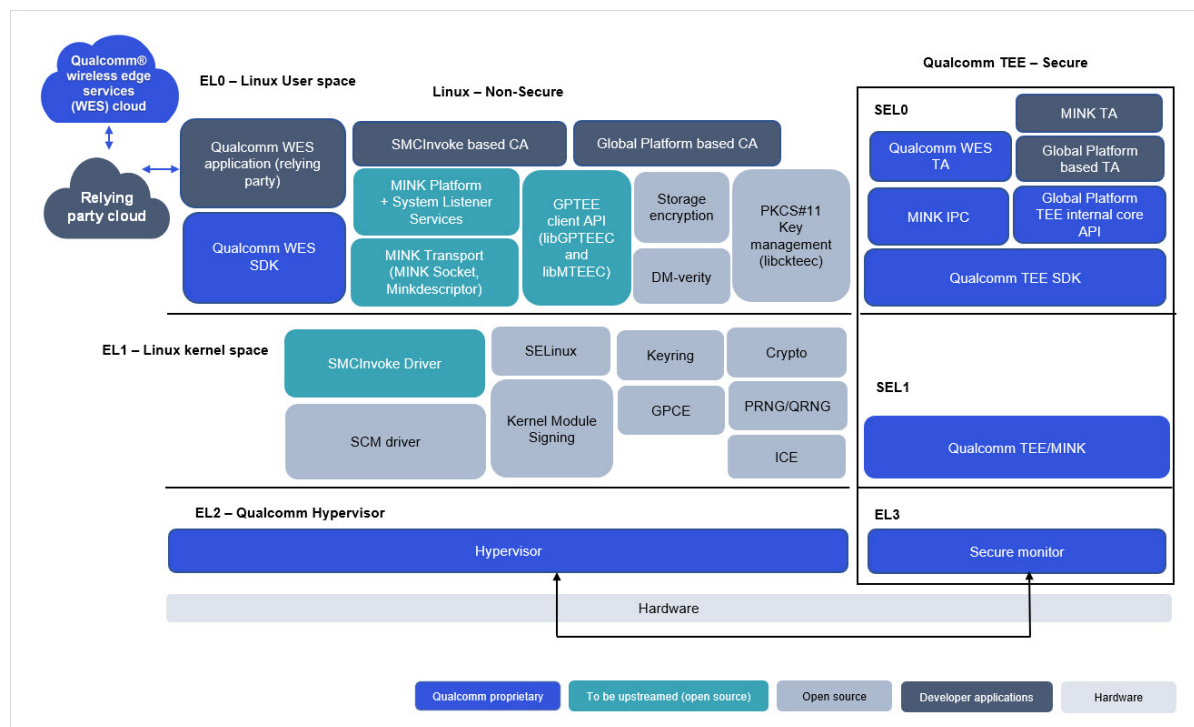


Figure 4-1 Qualcomm Linux security software components

The Qualcomm Linux security software has the following key components, each operating in different execution environments:

Exception levels	Software components
Non-secure EL0	Linux user space – Mink, key management, client applications, and Qualcomm WES
Non-secure EL1	Linux kernel space – SMC invoke and crypto/PRNG driver
Non-secure EL2	Qualcomm Hypervisor
Secure EL0	Qualcomm TEE SDK and trusted application
Secure EL1	Qualcomm TEE MINK
Secure EL3	Secure Monitor

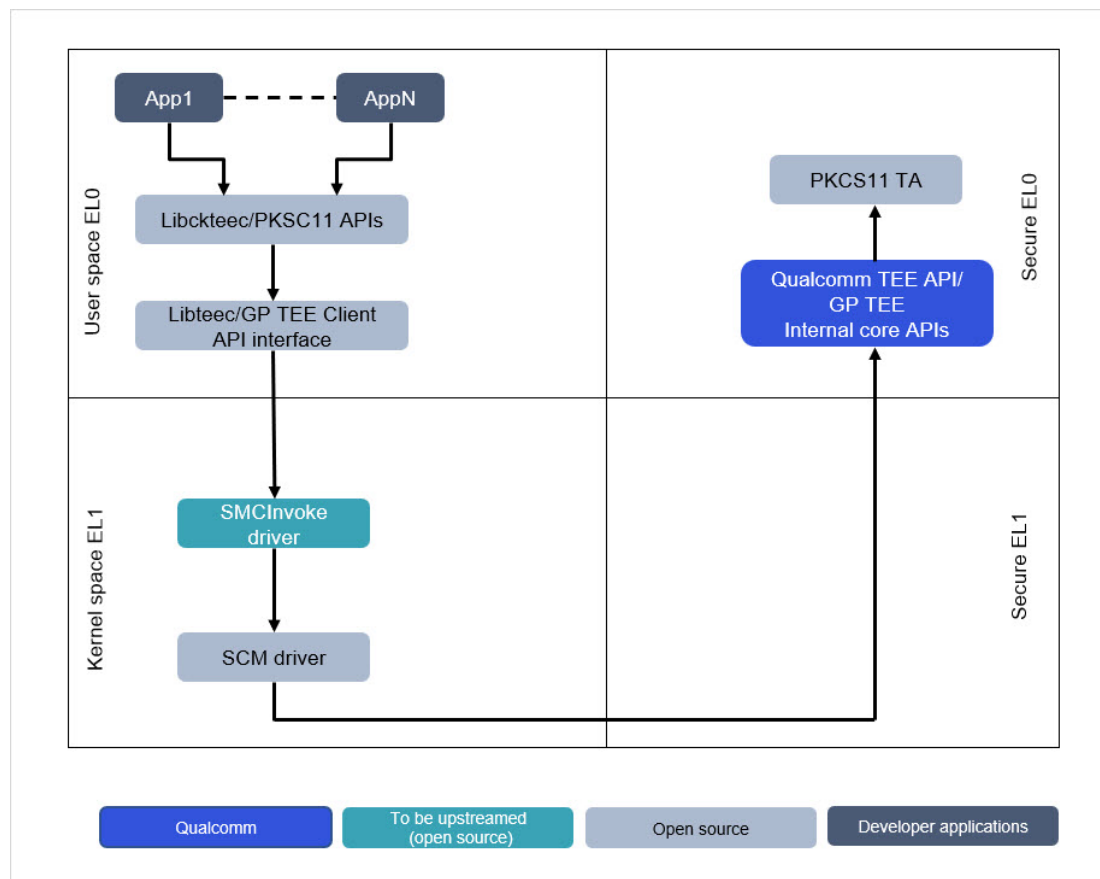
Linux user space (Non-secure EL0)

This includes the primary security modules/interfaces present in the Linux user space, which operates in non-secure EL0 as per the [Arm exception levels](#).

- MINK – Linux (MINK Platform/System Listener Services/Transport mechanism)
 - Provides the services and transport mechanism to use the Qualcomm TEE capabilities via TA.
 - Constitutes the system listener services, which are designed to extend the Qualcomm TEE functionalities. The MINK interface is also commonly known as [SMC invoke](#).

- Implements the GP TEE client APIs for executing GP-based TA through `libGPTEE` and `libMTEE` libraries. For more information, see <https://globalplatform.org/specs-library/tee-client-api-specification/>.
 - Implements client applications (CA) in a rich execution environment (Linux user space). It interacts with TA that operates in Qualcomm TEE. The various CAs include:
 - SMC invoke-based/MINK CA
 - CA that is developed using MINK/SMC invoke APIs
 - GP-based CA
 - CA that is developed using the GP-based TEE client API specification
- Key management
 - The crypto client library (`libckteec`) interfaces with the PKCS#11 interface of TEE. User space applications running on Linux can use this library for direct access to the PKCS#11 interface.
 - The `libckteec` library defines the interface between an application and a cryptographic device, enabling applications to treat cryptographic devices as tokens and perform cryptographic functions as implemented by these tokens.
 - The PKCS#11 interface provides a range of cryptographic services for encryption, decryption, signature generation, signature verification, and permanent key storage.

The figure illustrates key management in kernel and user space.



- See [Qualcomm WES](#)
- See [Storage encryption](#)

For more information on the various security software components, see [Features](#).

Linux – Kernel space (Non-secure EL1)

The following are the security-specific kernel drivers:

- SMC invoke driver:
 - Provides a Linux abstraction for Qualcomm TEE objects by encapsulating them in file descriptors.
 - Marshalls invocations into the Qualcomm TEE objects in buffers that are shared with Qualcomm TEE.
 - Tracks and dispatches callback invocations to Linux processes.
 - Keeps track of all the shared memory objects, providing information about them to Qualcomm TEE.
- For the CM and crypto/PRNG drivers, see the respective kernel documentation.

Qualcomm Hypervisor (Non-secure EL2)

See [Qualcomm Hypervisor](#).

Qualcomm TEE SDK (Secure EL0)

Qualcomm offers a software development kit (SDK) for developing and building TAs. This SDK includes the necessary build system, header files, and library dependencies to compile TA.

- Trusted applications. See [Qualcomm TEE](#).
- To develop the SMC invoke TA, GP TA, and IDL/Object-based Qualcomm TEE, service APIs are available for users who have full access to the proprietary software shipped with Qualcomm Linux. For more information, see [Qualcomm Linux Security Guide - Addendum](#).

Qualcomm TEE MINK (Secure EL1)

In a Qualcomm TEE kernel-based system, software operates in one or more communicating domains, such as the kernel domain or user domain.

These domains vary in their ability to access memory and other system resources.

- One of these domains, referred to as the kernel domain, executes at secure EL1 and has complete control over the system's resources.
- The other domains, known as user domains or processes, execute at secure EL0 and have restricted access to system resources. MINK allows for precise control over the access granted to a process.

An IPC primitive is an object capability-based synchronous message passing facility, which allows code executing in one domain to invoke objects running in other domains. The IPC MINK-primitives enable communication between the domains.

Secure monitor (Secure EL3)

The secure monitor, a component of TZ, is responsible for managing the transition between the secure and non-secure worlds.

It operates in Monitor mode, which is activated by executing the secure monitor call (SMC) instruction from a privileged Arm mode.

The secure monitor ensures accurate context saving and restoration when switching between the non-secure and secure worlds. Additionally, it handles the initial processing of secure interrupts.

5 APIs

The security APIs offer the ability to interface with the Linux kernel and the device's hardware. They also facilitate various software services that can be executed in a trusted execution environment.

User space APIs

These are functions that the Linux OS accesses to interact with the kernel.

Users with full access to the proprietary software shipped with Qualcomm Linux can view these APIs. If you have access, see [User space APIs](#).

Interfaces exposed for PKCS#11

See [PKCS11 Cryptographic Token Interface Usage Guide](#) and [PKCS #11 Cryptographic Token Interface Base Specification Version 2.40 \(oasis-open.org\)](#).

Kernel APIs

These are functions that allow the Qualcomm Linux software to interact with the device hardware.

Crypto APIs

The `qcrypto.ko` driver is located on the device at `/lib/modules/<version>/kernel/drivers/crypto/qce`. Both kernel-level and user-level APIs can access the crypto engine. For the APIs, see the kernel crypto documentation at `Documentation/crypto/`.

The following algorithms are supported:

- RFC 4309 (CCCM(AES))
- CCM (AES)
- Authenc (HMAC (SHA-256), CBC (AES))
- Authenc (HMAC (SHA-256), CBC (DES3_EDE))
- Authenc (HMAC (SHA-256), CBC (DES))
- Authenc (HMAC (SHA-1), CBC (DES3_EDE))
- Authenc (HMAC (SHA-1), CBC (DES))
- HMAC (SHA-256)
- HMAC (SHA-1)
- SHA-256
- SHA-1

- CBC (DES3_EDE)
- ECB (DES3_EDE)
- CBC (DES)
- ECB (DES)
- XTS (AES)
- CTR (AES)
- CBC (AES)
- ECB (AES)

For more information on the Qualcomm crypto core, see [Cryptography](#).

For more details on crypto APIs, see <https://www.kernel.org/doc/html/v6.6/crypto/index.html>.

Hardware random generator APIs

The Qualcomm Linux platform supports a true random number generator using the `qcom-rng` Linux driver. The random number generated from `qcom-rng` uses Kernel crypto for the random number generator API.

In user space, a random number can be accessed at `/dev/hwrng`. For more information on the hardware random number generator, see the kernel documentation at `Documentation/hw_random.txt`.

For PRNG APIs, see https://www.kernel.org/doc/Documentation/hw_random.txt.

Qualcomm TEE APIs

Qualcomm TEE provides a collection of APIs that offer services to secure applications. These services include heap management, logging, secure file system access, listener interactions, and cryptography and hashing functions.

Users with full access to the proprietary software shipped with Qualcomm Linux can view these APIs. If you have access, see [Qualcomm Linux Security Guide - Addendum → Qualcomm TEE APIs](#).

6 Develop

Users who have full access to the proprietary software shipped with Qualcomm Linux can develop and execute TA and CA. If you have access, see [Qualcomm Linux Security Guide - Addendum → Develop](#).

7 Tools

To develop the TZ components and TAs on Qualcomm TEE, use the Qualcomm® Snapdragon™ Mobile PC Platform LLVM compiler for Arm® Technology.

You can use Sectools v2 for signing the images, generating the fuse blower image for secure boot, and generating secure debug policies.

Build system

For more information on the toolchain, build process, and compilation, see [GitHub workflow \(firmware and extras\)](#).

Secure boot

The table lists the documentation for Sectools v2:

Document	Description
SecTools V2: Metabuild Secure Image User Guide	Perform the secure-image operations on metabuild software images.
SecTools V2: Fuse Blower User Guide	Create and sign the fuse blower images. When used by a device, a fuse blower image results in the specified fuse being blown.
SecTools V2: ELF Tool User Guide	Generate, add segments, and combine the ELF software images.
SecTools V2: MBN Tool User Guide	Add the modem configuration binary (MBN) headers to binary images.
SecTools V2: ELF Consolidator User Guide	Create the consolidated ELF software images. A consolidated ELF contains the contents of multiple subsystem images.
SecTools V2: Secure Image User Guide	Sign, encrypt, and inspect Qualcomm software images.
Sectools v2: Secure Debug User Guide	Generate and sign the debug policy images to enable device debugging and authentication.

8 Bring up

You can verify the status of various security features.

NOTE Remember to run all the SSH commands in the SELinux Permissive mode. The Enforcing mode will be supported in the future. For instructions on how to connect to the device, see [Qualcomm Linux Build Guide](#) → [How to](#) → [How to SSH](#).

Verify TZ/Device configuration/Hypervisor image loading

The XBL logs contain information about the TZ/Device configuration/Hypervisor image loading.

For example:

- **Device configuration (DEVCFG)**
 - B - 1206031 - QSEE Dev Config - Image Load, Start
 - D - 763 - Auth Metadata
 - D - 549 - Segments hash check
 - D - 13054 - QSEE Dev Config - Image Loaded, Delta - (53248 Bytes)
- **TZ**
 - B - 1228113 - QSEE - Image Load, Start
 - D - 26382 - Auth Metadata
 - D - 22234 - Segments hash check
 - D - 88999 - QSEE - Image Loaded, Delta - (4027792 Bytes)
- **Hypervisor**
 - B - 1402237 - QHEE - Image Load, Start
 - D - 26383 - Auth Metadata
 - D - 7045 - Segments hash check
 - D - 35258 - QHEE - Image Loaded, Delta - (1491024 Bytes)
- **APDP**
 - B - 978348 - APDP - Image Load, Start
 - D - 42212 - Auth Metadata
 - D - 458 - Segments hash check
 - D - 48434 - APDP - Image Loaded, Delta - (17332 Bytes)

Verify secure boot

The XBL logs contain the secure boot status of the device. The logs include information about the boot interface, secure boot status, boot configuration, JTAG ID, OEM ID, and serial number.

```
S - Format: Log Type - Time(microsec) - Message - Optional Info

S - Log Type: B - Since Boot(Power On Reset), D - Delta, S - Statistic

S - QC_IMAGE_VERSION_STRING=BOOT.MXF.1.0.c1-00037-KODIAKLA-1

S - IMAGE_VARIANT_STRING=SocKodiakLAA

S - OEM_IMAGE_VERSION_STRING=hu-apasunur-hyd

S - Boot Interface: USB
```

S - Secure Boot: On

```
S - Boot Config @ 0x00786070 = 0x000000c1

S - JTAG ID @ 0x00786130 = 0x001970e1

S - OEM ID @ 0x00786138 = 0x00000000

S - Serial Number @ 0x00786134 = 0x4172f1dd
```

Verify SELinux status

1. Verify the kernel configuration:

```
CONFIG_SECURITY_SELINUX=y
```

2. Connect to the device via SSH.

3. View the SELinux enable status and other details, by run the following `seinfo` commands:

```
Statistics for policy file: /sys/fs/selinux/policy
Policy Version:          33 (MLS enabled)
Target Policy:           selinux
Handle unknown classes:  allow
Classes:                 131   Permissions:          423
Sensitivities:           16   Categories:           1024
Types:                   4376  Attributes:            319
```

4. Verify the SELinux enforce status from the console or connect to the device via SSH:

```
$ getenforce
enforcing - if selinux is set to enable
```

Verify PIL image loading – Sample logs

WLAN(remoteproc1)	730, 0x000000000000459B4 8.700828: remoteproc remoteproc1: Remote processor 8a00000. Remoteproc is now up
cDSP(remoteproc3)	735, 0x000000000000465A3 8.794052: remoteproc remoteproc3: Remote processor a300000. Remoteproc is now up
aDSP(remoteproc2)	741, 0x000000000000469FC 8.828033: remoteproc remoteproc2: Remote processor 3000000. remoteproc is now up
MODEM(remoteproc0)	1035, 0x0000000000006B78B 13.433955: remoteproc remoteproc0: Remote processor 4080000. Remoteproc is now up
A660_zap	sh-5.1# dmesg grep gfx [7.822629] kgs1-iommu SoC@0:QCOM, kgs1-iommu@3da0000:gfx3d_user: Adding to iommu group 15 [7.870446] kgs1-3d 3d00000.qcom, kgs1-3d0: bound SoC@0:QCOM, kgs1-iommu@3da0000:gfx3d_user (ops kgs1_mmu_cb_component_ops [msm_kgs1])
Video (Vpu20_1v.mbn)	sh-5.1# dmesg grep video [7.094229] videodev: Linux video capture interface: v2.00 [7.847469] qcom-iris aa00000.video-codec: Adding to iommu group 17 [7.856647] qcom-iris aa00000.video-codec: no reset clocks found [10.131119] [drm] [msm-dsi-warn]: [nt36672e LCD video mode DSI novatek panel with DSC] fall back to default te-pin-select [10.188079] [drm:dsi_display_bind [msm_drm]] [msm-dsi-info]: Successfully bind display panel 'QCOM, mdss_dsi_nt36672e_fhd_plus_120_video '

Verify Qualcomm TEE mink/GPTEE API availability

1. Verify the availability of the library on the device:

```
/usr/lib/
libmink*
libGPMTEE*
libGPTEE*
```

2. Verify if the mink listener service is running:

```
ps -ef | grep qtee_suppllicant and qtee_suppllicant is running
ps -ef | grep ssgtzd
```

Verify SMC invoke driver status

1. Connect to the device via SSH.
2. Verify if the `/dev/smcinvoke` node is present:

```
ls -l /dev/smcinvoke
```

Verify Qualcomm WES status

Users with full access to the proprietary software shipped with Qualcomm Linux can verify the Qualcomm WES status. If you have access, see [Qualcomm Linux Security Guide - Addendum → Bring up](#).

Check RPMB provision status

Connect to the device via SSH and run the following command:

```
sh-5.1# rpmbClient smci -p 1
```

The following message will be displayed:

```
----- SMCINVOKE INTERFACE
WARNING!!! You are about to provision the RPMB key. This is a ONE time
operation and CANNOT be reversed.
----- 0 -> Provision
Production key 1 -> Provision Test key 2 -> Check RPMB key provision status
----- Select an option to
proceed: 2 RMPB Key status: RPMB_KEY_NOT_PROVISIONED (f)
```

9 Configure

Qualcomm TEE configurations can be adjusted via the device configuration (devcfg) framework.

NOTE Remember to run all the SSH commands in the SELinux Permissive mode. The Enforcing mode will be supported in the future. For instructions on how to connect to the device, see [Qualcomm Linux Build Guide → How to → How to SSH](#).

TrustZone offers configuration options through XML files such as the `trustzone_images/ssg/securemsm/trustzone/qsee/mink/oem/config/<chipset>/oem_config.xml` file.

The XML configuration files are built into `devcfg.mbn`. The following command is used to compile the devcfg image from TZ.XF.5.0:

```
cd trustzone_images/build/ms$python3 build_all.py -b TZ.XF.5.0
CHIPSET=<chipset> devcfg
devcfg.mbn image is generated under below location
trustzone_images/build/ms/bin/<build_flavor>
```

For instructions on building and compiling, see [Qualcomm Linux Build Guide → GitHub workflow \(firmware and extras\)](#).

Device configuration

Property	Description
<code>OEM_pil_secure_app_load_region_size</code>	Configuration to customize TA size.
<code>OEM_pil_subsys_load_region_start</code>	Configuration to customize the PIL load start address when there is any change from the default memory map.
<code>OEM_pil_subsys_load_region_size</code>	Configuration to customize the PIL size when there is any change from the default memory map.
<code>OEM_enable_app_fatal_err</code>	This property can be used to force a TZ system to fatal error when a specific TA crashes. Used with <code>OEM_crash_ta_name</code> .
<code>OEM_crash_ta_name</code>	Replaces the entry with the TA name that crashed and the TA on which the secure kernel is expected to crash.
<code>OEM_sec_wdog_bark_time</code>	Changes the default configuration of the device for secure watchdog bark time.
<code>OEM_sec_wdog_bite_time</code>	Changes the default configuration of the device for secure watchdog bite time.
<code>OEM_tz_log_level</code>	Set the TZ log level: ▪ Fatal: 0 ▪ Error: 1 ▪ Debug: 2

RPMB-based SFS antirollback protection

The RPMB-based SFS antirollback protection can be enabled or disabled in the XML configuration file located at: `trustzone_images/ssg/securemsm/trustzone/qsee/mink/oem/config/common/ cmnlib_oem_config.xml`

<code>cmnlib_gppo_rpmb_enablement</code>	▪ Enabled (Default value. Change only when required.) ▪ Disabled
--	--

SELinux configuration

1. Connect to the device via SSH.
2. Change the SELinux mode by using the following commands.
 - To switch the device to Enforcing mode:

```
setenforce 1
```
 - To switch the device to Permissive mode:

```
setenforce 0
```
 - To check the current configuration of the device (Enforcing or Permissive mode):

```
getenforce
```

9.1 Enable secure boot

Secure boot is enabled by blowing a set of hardware fuses that are part of QFPROM. The hash of the root certificate is blown into the hardware fuse, which serves as the primary RoT.

See the workflow to understand the off-target preparation and the on-device execution.



Secure boot is guaranteed only after blowing a QFPROM, which is an eFuse. The eFuse configuration required to enable platform secure boot includes:

- Enabling image authentication and antirollback protection.
- Disabling Debug and JTAG access.
- Blowing *disable read-write permission* fuses for the QFPROM regions.

To enable secure boot, follow these steps:

1. Obtain the unique OEM ID from Qualcomm.
The ID is required when using the code authorization signing services (CASS) or Qualcomm WES services. Alternatively, you can use 0 as the value for the OEM ID.
2. Generate and configure signing assets such as keys and certificates.
3. Generate a signed ELF (sec.elf) for blowing fuses.
4. Sign firmware images.
5. Flash sec.elf and signed images to the device.

Images that are not based on Linux are signed using Sectools v2 and a local signer. This process requires signing certificates and keys to be present on the local machine where the signing takes place. However, these keys are not secure and can be exposed during and after the signing process on the machine running Sectools.

NOTE Qualcomm does not provide guidance on how to secure these keys. It is recommended to follow standard security protocols, obtain keys/certificates from a trusted certification authority (CA), and protect these keys using a hardware security module (HSM).

Keys and certificates that are self-generated using the OpenSSL tool do not have a link to any certificate authority.

Prerequisites to enable secure boot

1. Set up the environment variable for Sectools v2. For more information, see [Tools](#).

The following code snippets are provided as references. Replace `<chipset>` and `<chipset>.LE.X.x` according to the build in use. For example, `<QCM6490.LE.1.0>`.

NOTE The terms metabuild, meta, and meta paths are used interchangeably to refer to the path from the Qualcomm ChipCode™ Portal. The metabuild denotes the complete Qualcomm ChipCode release for a build.

- For Windows:

```
set SECTOOLS=\<chipset>.LE.X.x\common\sectoolsv2\ext\windows
\sectools.exe
```

- For Linux:

```
setenv SECTOOLS=/\<chipset>.LE.X.x/common/sectoolsv2/ext/linux/sectools

export SECTOOLS=/\<chipset>.LE.X.x/common/sectoolsv2/ext/linux/sectools
```

2. Install Sectools v2, which is located in the meta at `<chipset>.LE.X.x/common/sectoolsv2/ext/<platform>`
 - The `<chipset>_security_profile.xml` file is located in the meta at `<chipset>.LE.X.x/ common/sectoolsv2`
 - The minimum version required is 1.17.
3. It is recommended to use a hardware security module (HSM). However, if a LOCAL (insecure) signer is used, see the LOCAL signing option with `–signing-help` in [SecTools V2: Secure Image User Guide](#).
4. All the keys and certificate generation commands are executed using OpenSSL 1.1.1g (21 Apr 2020). It is a prerequisite to install OpenSSL.
5. Ensure that you enable secure boot on a device that is not RPMB provisioned. See [Check RPMB provision status](#).

RPMB will be automatically provisioned with production keys after secure boot fuses are blown.

9.1.1 QFPROM fuses

The table lists the various QFPROM fuse values that can be blown to enable the secure boot.

Fuse name	Start address	Bit number	Fuse blow value	Description
Read permissions				
Secondary Key derivation Key Read disable	7801A8	24	1	After provisioning the SKDK, blow this bit to secure the secondary key from being read back. A secure path hardware exists from SKDK to the crypto engine.
Write permissions				
Read permissions write disable	7801B0	6	1	Blow this bit after the region has been provisioned to disable further QFPROM changes to this region.
FEC enables write disable	7801B0	8	1	Blow this bit after the region has been provisioned to disable further QFPROM changes to this region.
OEM configuration write disable	7801B0	9	1	Blow this bit after the region has been provisioned to disable further QFPROM changes to this region.
Public key hash 0 write disable	7801B0	17	1	Blow this bit after the region has been provisioned to disable further QFPROM changes to this region.
OEM secure boot write disable	7801B0	23	1	Blow this bit after the region has been provisioned to disable further QFPROM changes to this region.
Secondary key derivation key write disable	7801B0	24	1	Blow this bit after the region has been provisioned to disable further QFPROM changes to this region.
FEC enable				
OEM secure boot FEC enable	7801B8	23	1	Blow this bit to enable FEC for OEM secure boot region. Ensure that the complete region is provisioned before FEC is enabled.
Secondary key derivation key FEC enable	7801B8	24	1	Blow this bit to enable FEC for the secondary KDF key. Ensure that the complete region is provisioned before FEC is enabled.
OEM Config				
WDOG_EN	7801C0	14	1	This prevents the WDOG_DISABLE GPIO from disabling WDOG, freeing up the GPIO and preventing potential abuse by an attacker.

Fuse name	Start address	Bit number	Fuse blow value	Description
SHARED_QSEE_SPIDEN_DISABLE	7801C0	30	1	This is a shared Qualcomm TEE secure invasive debug disable bucket. A corresponding Qualcomm fuse can override this OEM-controlled fuse.
SHARED_QSEE_SPNIDEN_DISABLE	7801C0	31	1	This is a shared Qualcomm TEE secure non-invasive debug disable bucket. A corresponding Qualcomm fuse can override this OEM-controlled fuse.
SHARED_MSS_DBGGEN_DISABLE	7801C4	32	1	This is a shared MSS invasive debug disable bucket. A corresponding Qualcomm fuse can override this OEM-controlled fuse.
SHARED_MSS_NIDEN_DISABLE	7801C4	33	1	This is a shared MSS non-invasive debug disable bucket. A corresponding Qualcomm fuse can override this OEM-controlled fuse.
SHARED_CP_DBGGEN_DISABLE	7801C4	34	1	This is a shared CP invasive debug disable bucket. A corresponding Qualcomm fuse can override this OEM-controlled fuse.
SHARED_CP_NIDEN_DISABLE	7801C4	35	1	This is a shared CP non-invasive debug disable bucket. A corresponding Qualcomm fuse can override this OEM-controlled fuse.
SHARED_NS_DBGGEN_DISABLE	7801C4	36	1	This is a shared CP non-invasive debug disable bucket. A corresponding Qualcomm fuse can override this OEM-controlled fuse.
SHARED_NS_NIDEN_DISABLE	7801C4	37	1	This is a shared CP non-invasive debug disable bucket. A corresponding Qualcomm fuse can override this OEM-controlled fuse.
APPS_DBGGEN_DISABLE	7801C4	38	1	Blow this bit for a secure solution. This disables the application processor's global invasive debug capabilities (JTAG and monitor mode). The <code>OVERRIDE</code> registers can override this configuration.

Fuse name	Start address	Bit number	Fuse blow value	Description
APPS_NIDEN_DISABLE	7801C4	39	1	Blow this bit for a secure solution. This disables the application processor's global non-invasive debug capabilities (trace and performance monitoring). This can be overridden with the OVERRIDE registers.
SHARED_MISC_DEBUG_DISABLE	7801C4	40	1	This is a shared miscellaneous debug disable bucket. A corresponding Qualcomm fuse can override this OEM-controlled fuse.
EKU_ENFORCEMENT_EN	7801C8	30	1	Blowing this fuse enables enforcement of the EKU field in the certificate.
OEM_HW_ID[0:15]	7801CC	[32:47]	0	This represents the OEM hardware ID. Bits 15:0.
OEM_PRODUCT_ID[0:15]	7801CC	[48:63]	0	This represents the OEM product ID. Bits 15:0.
ANTI_ROLLBACK_FEATURE_EN[0]	7801D4	32	1	- Bit 0 - BOOT_ANTI_ROLLBACK_EN - Bit 1 - TZAPPS_ANTI_ROLLBACK_EN - Bit 2 - PILSUBSYS_ANTI_ROLLBACK_EN - Bit 3 - MSA_ANTI_ROLLBACK_EN
ANTI_ROLLBACK_FEATURE_EN[1]	7801D4	33	1	
ANTI_ROLLBACK_FEATURE_EN[2]	7801D4	34	1	
ANTI_ROLLBACK_FEATURE_EN[3]	7801D4	35	1	
PK hash				
PK hash 0[0:383]	780248	[0:383]	-	This is the OEM-specific root certificate PK hash value.
OEM secure boot				
OEM_SECURE_BOOT1_PK_HASH_IN_FUSE	780728	4	1	When this bit is '1', use the value stored in OEM_PK_HASH for the root certificate hash.
OEM_SECURE_BOOT1_AUTH_EN	780728	5	1	Blow this bit to enable secure boot for apps and other peripheral images. When this bit is '1', it enables authentication for any code that references secure boot configuration 1.

Fuse name	Start address	Bit number	Fuse blow value	Description
OEM_SECURE_BOOT2_PK_HASH_IN_FUSE	780728	12	1	For boot configuration 2: If this bit is '0', use the internal ROM hash index and OEM_SECURE_BOOT1_ROM_PK_HASH_IDX[3:0] for the root certificate hash. If this bit is '1', use the value stored in OEM_PK_HASH for the root certificate hash.
OEM_SECURE_BOOT2_AUTH_EN	780728	13	1	Blow this bit to enable a secure boot. When this bit is '1', it enables authentication for any code that references secure boot
OEM_SECURE_BOOT3_PK_HASH_IN_FUSE	780728	20	1	For boot configuration 3: If this bit is '0', use the internal ROM hash index and OEM_SECURE_BOOT1_ROM_PK_HASH_IDX[3:0] for the root certificate hash. When this bit is '1', use the value stored in OEM_PK_HASH for the root certificate hash.
OEM_SECURE_BOOT3_AUTH_EN	780728	21	1	Blow this bit to enable secure boot. When this bit is '1', it enables authentication for any code that references secure boot configuration 3.

Fuse name	Start address	Bit number	Fuse blow value	Description
Sec Key derivation Key				
Sec Key derivation Key[0:255]	780738	[0:255]		This 256-bit value is used as the secondary key derivation input, which is used to generate the secondary key for the crypto engine. When running in an insecure mode (no secure boot or Debug enabled), the SKDK is fed into the key derivation function to generate a unique non-secure secondary key for use by the crypto engine. When running in a secure mode (secure boot and debug disabled), the SKDK is fed directly to the crypto engine as the secondary key. After the SKDK value has been correctly programmed, the SKDK Read Disable must be blown to permanently protect the SKDK value. Prior to this correction, the software read the SKDK value from the QFPROM. The SBL fuse blow API can automatically generate a random number for use as the SKDK, ensuring that the SKDK value is never available outside of the device.

9.1.2 Generate local (insecure) root key and certificates

The version 3 (v3 and v3_attest) extensions are used to define the format for certificates and create CAs.

1. Create `opensslroot.cfg`, see [Sample OpenSSL configuration](#).
2. Create `v3.ext` and `v3_attest.ext`, use the following extensions.
 - `v3.ext`: This can be found at [/docs/manmaster/man5/x509v3_config.html \(openssl.org\)](#), and include the following settings:

```
authorityKeyIdentifier=keyid,issuer
subjectKeyIdentifier=hash
basicConstraints=CA:true,pathlen:0
keyUsage=keyCertSign
```
 - `v3_attest.ext`: This can be found at [/docs/manmaster/man5/x509v3_config.html \(openssl.org\)](#), and include the following settings:

```
authorityKeyIdentifier=keyid,issuer
basicConstraints=CA:FALSE,pathlen:0
```

```
keyUsage=digitalSignature
extendedKeyUsage=codeSigning
```

3. Prepare the environment, create a folder named OEM-KEYS to generate all certificates and keys at one location.

- For Linux, use the following commands:

```
cd /path/to/sectools/$ mkdir ./OEM-KEYS
cp /download/opensslroot.cfg ./OEM-KEYS
cp /download/v3.ext ./OEM-KEYS
cp /download/v3_attest.ext ./OEM-KEYS
```

- For Windows, copy opensslroot.cfg, v3.ext, and v3_attest.ext to the OEM-KEYS folder.

The algorithms supported are:

- Rivest–Shamir–Adleman (RSA) signature algorithm
- Elliptical curve digital signature algorithm (ECDSA)

9.1.3 Generate ECDSA root key and certificate

ECDSA offers superior security and performance compared to the RSA signature algorithm. As a result, the default configuration in Sectools supports ECDSA signing.

With ECDSA, two types of keys are created:

- The public key, which is accessible to everyone.
- The private key, which is only known to the owner of the key pair

NOTE To avoid copying any special characters along with the commands, it is recommended to manually type the commands on the command prompt or a notepad before execution.

You can modify and run the following ECDSA-specific commands to generate the root key and certificate:

1. Navigate to the OEM-KEYS directory and generate the ECDSA root key and certificate:

```
cd ./OEM-KEYS
```

```
openssl ecparam -genkey -name secp384r1 -outform PEM -out qpsa_rootca.key
openssl req -new -key qpsa_rootca.key -sha384 -out rootca_pem.crt -subj '/
C=US/CN=Generated OEM Root CA/OU=CDMA Technologies/OU=General Use OEM Key
(OEM should update all fields)/L=San Diego/O=SecTools/ST=California' -
config opensslroot.cfg -x509 -days 7300 -set_serial 1

openssl x509 -in rootca_pem.crt -inform PEM -out qpsa_rootca.cer -outform
DER
```

2. Generate the intermediate certificate authority (CA) key pair and certificate:

```
openssl ecparam -genkey -name secp384r1 -outform PEM -qpsa_attestca.key
openssl req -new -key qpsa_attestca.key -out ca.CSR -subj '/C=US/
ST=California/CN=Generated OEM Attestation CA/O=SecTools/L=San Diego' -
config opensslroot.cfg -sha384

openssl x509 -req -in ca.CSR -CA rootca_pem.crt -CAkey qpsa_rootca.key -
out ca_pem.crt -set_serial 1 -days 7300 -extfile v3.ext -sha384 -
CAcreateserial

openssl x509 -inform PEM -in ca_pem.crt -outform DER -out qpsa_attestca.cer
```

9.1.4 Generate RSA CA key pair and certificate

RSA is an encryption algorithm that uses a pair of keys to encrypt and decrypt data, ensuring secure data transmission.

With RSA, a private and a public key are created.

- The public key is accessible to anyone.
- The private key is only known to the owner of the key pair.

Either the public or private key can encrypt the data, and the other key decrypts it.

1. To generate the root CA key pair and certificate, run the following commands:

The key size used is 2048, however a key size of 4096 is also supported.

```
OpenSSL genrsa -out qpsa_rootca.key 2048

OpenSSL req -new -sha256 -key qpsa_rootca.key -x509 -out rootca_pem.crt -
subj /C=US/ST=California/L="San Diego"/OU="General Use Test Key (for
testing 13 only)"/OU="CDMA Technologies"/O=QUALCOMM/CN="QCT Root CA 1" -
days 7300 -set_serial 1 -config opensslroot.cfg -sigopt
rsa_padding_mode:pss -sigopt rsa_pss_saltlen:-1 -sigopt digest:sha256

OpenSSL x509 -in rootca_pem.crt -inform PEM -out qpsa_rootca.cer -outform
DER

OpenSSL x509 -text -inform DER -in qpsa_rootca.cer
```

2. To generate the attestation CA key pair and certificate, run the following commands using RSA with a key size of 2048:

```
openssl genrsa -out qpsa_attestca.key 2048

OpenSSL req -new -key qpsa_attestca.key -out attestca.csr -subj /C=US/
ST=CA/L="San Diego"/OU="CDMA Technologies"/O=QUALCOMM/CN="QUALCOMM
Attestation CA" -days 7300 -config opensslroot.cfg

OpenSSL x509 -req -in attestca.csr -CA rootca_pem.crt -CAkey
qpsa_rootca.key -out attestca_pem.crt -set_serial 5 -days 7300 -extfile
v3.ext -sha256 -sigopt rsa_padding_mode:pss -sigopt rsa_pss_saltlen:-1 -
sigopt digest:sha256

openssl x509 -inform PEM -in attestca_pem.crt -outform DER -out
qpsa_attestca.cer
```

9.1.5 Generate SHA-384 hash for RSA and ECDSA

The SHA-384 hash is crucial in cryptographic applications for several reasons, including enhancing security strength, creating digital signatures, ensuring compliance with standards, and future-proofing.

1. To generate the SHA-384 hash of the root certificate, run the following command:

```
openssl dgst -sha384 qpsa_rootca.cer >sha384rootcert.txt
```

2. Use Sectools to create the leaf (also known as the attestation) certificate, which is the final certificate in the chain).

9.1.6 Sign images

Image signing ensures the authenticity, integrity, and origin of the images.

To flash the images, do the following:

1. For using different signing methods and the secure image functionality, see [SecTools V2: Secure Image User Guide](#).
2. If the local signer is used and keys and certificates are generated using these commands, see [Generate local \(insecure\) root key and certificates](#).
3. To sign a single image, run the following command, where `tz.mbn` is used as an example.

NOTE You should replace the values highlighted in **bold as** according to your requirement.

```
<meta>/common/sectoolsv2/ext/<platform>/Sectools.exe secure-image --sign /  
path/to/tz.mbn --image-id=TZ --security-profile <meta>/common/sectoolsv2/  
<chipset>_security_profile.xml --oem-id=0x1 --oem-product-id=0xabcd --anti-  
rollback-version=0x0 --signing-mode=LOCAL --root-certificate=./OEM-KEYS/  
qpsa_rootca.cer --ca-certificate=./OEM-KEYS/qpsa_attestca.cer --ca-key=./  
OEM-KEYS/qpsa_attestca.key --outfile ./signed_images_out/tz.mbn
```

4. For the images that should be split, use the `--pil-split` option.
5. For signing the complete metabuild, see the commands from [SecTools V2: Metabuild Secure Image User Guide](#).

9.1.7 Generate signed sec.elf image

This information provides sample commands only. For generating fuse blower binary, see [SecTools V2: Fuse Blower User Guide](#).

The following are sample commands for Sectools on Windows:

NOTE You can replace the values highlighted in **bold as** per your requirement.

- **Stage 1: Basic secure boot (Image authentication + OEMID + MODEL ID)**

Run the following command:

```
<meta>/common/sectoolsv2/ext/<platform>/sectools.exe fuse-blower --
security-profile

<meta>/common/sectoolsv2/<chipset>_security_profile.xml --fuse-pk-
hash-0=0xf953644308944bb811ca0ec2a736a17fe38509941ce7f55860130857813c8378e9
3359b70dfd874c270dca08a53bd99f --fuse-oem-secure-boot1-pk-hash-in-fuse --
fuse-oem-secure-boot1-auth-en --fuse-oem-secure-boot2-pk-hash-in-fuse --
fuse-oem-secure-boot2-auth-en --fuse-oem-secure-boot3-pk-hash-in-fuse --
fuse-oem-secure-boot3-auth-en --fuse-oem-hw-id=0x0001 --fuse-oem-product-
id=0xabcd --generate --sign --signing-mode=LOCAL --root-certificate=./OEM-
KEYS/qpsa_rootca.cer --ca-certificate=./OEM-KEYS/qpsa_attestca.cer --ca-
key=./OEM-KEYS/qpsa_attestca.key --oem-id=0x1 --oem-product-id=0xabcd --
outfile basic_sec.elf
```

- **Stage 2: Complete secure boot (Basic secure boot + Debug disable + Arb + write permission disable):**

Run the following command:

```
<meta>/common/sectoolsv2/ext/<platform>/sectools.exe fuse-blower --
security-profile <meta>/common/sectoolsv2/<chipset>_security_profile.xml --
fuse-pk-
hash-0=0xf953644308944bb811ca0ec2a736a17fe38509941ce7f55860130857813c8378e9
3359b70dfd874c270dca08a53bd99f --fuse-oem-secure-boot1-pk-hash-in-fuse --
fuse-oem-secure-boot1-auth-en --fuse-oem-secure-boot2-pk-hash-in-fuse --
fuse-oem-secure-boot2-auth-en --fuse-oem-secure-boot3-pk-hash-in-fuse --
fuse-oem-secure-boot3-auth-en --fuse-oem-secure-boot-fec-enable --fuse-
wdog-en --fuse-shared-qsee-spiden-disable --fuse-shared-qsee-spniden-
disable --fuse-shared-mss-dbgen-disable --fuse-shared-mss-niden-disable --
fuse-shared-cp-dbgen-disable --fuse-shared-cp-niden-disable --fuse-shared-
ns-dbgen-disable --fuse-shared-ns-niden-disable --fuse-apps-dbgen-disable
--fuse-apps-niden-disable --fuse-shared-misc-debug-disable --fuse-eku-
enforcement-en --fuse-anti-rollback-feature-en=0xF --fuse-sec-key-
derivation-key=0x00 --fuse-read-permissions-write-disable --fuse-oem-
configuration-write-disable --fuse-secondary-key-derivation-key-read-
disable --fuse-public-key-hash-0-write-disable --fuse-oem-secure-boot-
write-disable --fuse-secondary-key-derivation-key-write-disable --fuse-
secondary-key-derivation-key-fec-enable --fuse-fec-enables-write-disable --
generate --sign --fuse-oem-hw-id=0x0001 --fuse-oem-product-id=0xabcd --
signing-mode=LOCAL --root-certificate=./OEM-KEYS/

qpsa_rootca.cer --ca-certificate=./OEM-KEYS/qpsa_attestca.cer --ca-key=./
OEM-KEYS/qpsa_attestca.key --oem-id=0x1 --oem-product-id=0xabcd --outfile
sec.elf
```

9.1.8 Flash images

Flashing images involves writing an entire image, including partitions, file systems, and data, onto a storage device.

To flash the images, follow these steps:

1. See [QFPROM fuses](#) for the list of fuses to configure.
2. Replace all the binaries with the signed binaries including `prog_firehose_ddr.elf`.
3. To flash all the signed binaries to the device, see [Build from source → Flash](#).
4. After generating the signed images and `sec.elf`, enable secure boot:
 - a. Flash the signed images first without `sec.elf` and ensure that the device boots successfully.
 - b. Flash the signed images and `sec.elf` using the flash procedure from [Build from source → Flash](#).
 - c. Verify that the secure boot is enabled using [Verify secure boot](#).
5. When the secure boot is enabled, the device expects images to be flashed using a secure programming method called validated image programming (VIP). In this release, you can proceed with flashing the images on the secure device by disabling VIP using the following workaround programmer (`prog_firehose_ddr.elf`) image at: `<>/BOOT.MXF.1.0.c1\boot_images\boot\QcomPkg\Library\DevPrgLib\devprg_transfer.c`

6. Set the `vip->state` to `VIP_DISABLED` irrespective of the secure boot enable check in the following function:

```
int devprg_transfer_init(void)
{
    int secboot, result;
    struct vip_data *vip = &vip_data;

    devprg_init_vip_state();
    secboot = devprg_is_secure_boot_enabled();
    // if (secboot == 0) /*comment this to set vip state to VIP_DISABLED
        vip->state = VIP_DISABLED;

    result = devprg_transport_init();

    return result;
}
```

7. To rebuild `prog_firehose_ddr.elf`, see [GitHub workflow \(firmware and extras\) → Build firmware → Build Boot](#).
8. If any of the PIL signed images are not flashed using PCAT, follow these steps to push the PIL images manually using SCP:

```
push adsp, cdsp, modem, wlan, ipa pil split binaries
```

For instructions, see [How to → How to SSH](#).

- a. Copy and replace the PIL split bins and the .mdt files generated in the signed output to the <<QCM6490.LE.x.x>\common\build\ufs\bin\QCM6490_fw\lib\firmware\qcom\qcm6490\ folder.

- b. Connect to the device as the root via SSH. For instructions, see [How to → How to SSH](#).

Run the following command:

```
mount -o rw,remount /
scp <<QCM6490.LE.x.x>\common\build\ufs\bin\QCM6490_fw\lib\firmware\qcom\qcm6490\ . root@<IP_address>:/lib/firmware/qcom/qcm6490/
```

Push gfx (a660_zap) pil split binaries, a660_zap.mdt and a660_zap.mbn from signed output

```
scp <a660_zap signed output folder>\. root@<IP_address>:/lib/firmware/
Push signed Venus binary:
```

```
scp vpu20_1v.mbn root@<IP_address>:/lib/firmware/qcom/vpu-2.0/
```

```
reboot
```

9.2 Enable SELinux

When SELinux is enabled, all system objects, including files, directories, processes, sockets, drivers, and more, are labelled with a security context.

A security context consists of a user, role, type identifier, and optional sensitivity, separated by colons.

For example: `user:role:type:sensitivity`

Note that *User* is unrelated to a Linux user, and *type* is unrelated to what kind of object it is.

- A set of valid users, roles, and types is defined in the policy.
- Different objects can be labeled with the same security context.
- The MAC mechanism of SELinux security policies is implemented using:
 - Type enforcement (TE)
 - Role-based access control (RBAC)
 - Multilevel security (MLS)
- Types enable the policy to specify the allowed operations.

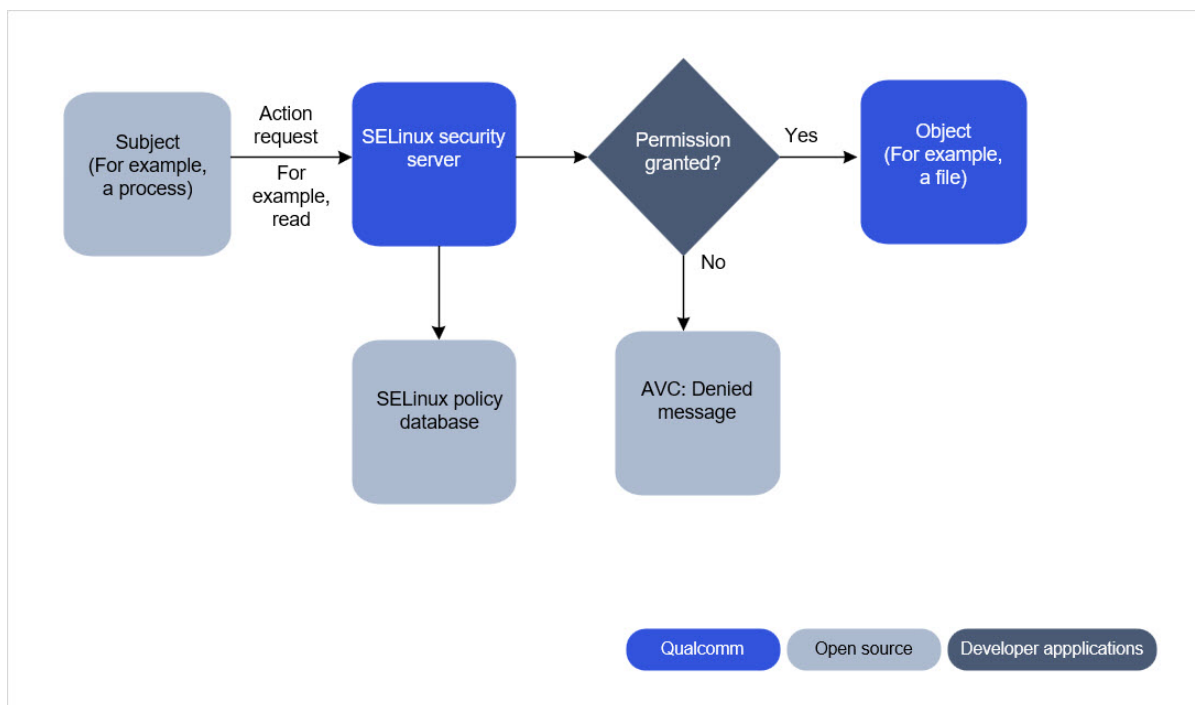


Figure 9-1 SELinux process

The following procedure explains how to enable SELinux and change the SELinux mode.

NOTE For testing, the SELinux implementation is launched in the Permissive mode, rather than the non-functional Disabled mode or the most stringent Enforcing mode. This mode acts as a reference and facilitates testing and development

1. To enable or disable SELinux:

- To disable SELinux for the build, delete the lines of code at [SELINUX_DISTRO_SECTION](#). By default, SELinux is enabled for the build.
- To enable SELinux, add the lines of code at [SELINUX_DISTRO_SECTION](#) as shown in the figure to enable SELinux
- Use policy version 33.
- To add policies for SELinux, see upstream [refpolicy](#).

The figure shows the steps in a SELinux:

```

#Selinux support
DISTRO_FEATURES_FILTER_NATIVE:append = " selinux"
DISTRO_FEATURES_FILTER_NATIVESDK:append = " selinux"
DISTRO_EXTRA_RDEPENDS:append = " ${@bb.utils.contains('DISTRO_FEATURES','selinux','packagegroup-selinux-minimal','','d'))"
PREFERRED_PROVIDER_virtual/refpolicy = "refpolicy-mls"
DEFAULT_ENFORCING = "permissive"
SELINUX_FILE_CONTEXTS_EXT4 = "-f ${IMAGE_ROOTFS}/etc/selinux/mls/contexts/files/file_contexts"
EXTRA_IMAGECMD:ext4 += " ${@bb.utils.contains('DISTRO_FEATURES','selinux','${SELINUX_FILE_CONTEXTS_EXT4}','','d'))"
  
```

2. Check the system status with `getenforce` on target. This command returns one of three values:
 - enforcing
 - permissive
 - disabled

3. To change the mode, select a mode at runtime by running `setenforce` with a number (this change will not persist after reboot).

Command	Result
<code>setenforce 1</code>	Move to Enforcing mode
<code>setenforce 0</code>	Move to Permissive mode

- a. To persist after reboot:

- i. Connect to the device via SSH. For instructions, see [How to → How to SSH](#).
- ii. Run the following commands:

```
umount -l / etc
mount -o rw,remount /
```

- iii. Edit `SELINUX=` to one of the three supported values: `enforcing`, `permissive`, or `disabled` in `/etc/selinux/config`.

- iv. Reboot the device with using the following command:

```
reboot
```

- b. To specify the SELinux mode in the build: Change the `DEFAULT_ENFORCING` build flag to one of the three supported values: `enforcing`, `permissive`, or `disabled`.

```
conf/distro/include/qcom-base.inc
-- DEFAULT_ENFORCING = "permissive"
++ DEFAULT_ENFORCING = "enforcing"
```

4. The SELinux Disabled mode leaves behind many code paths that go through the SELinux framework. These code paths are not useful for KPI testing or verifying bugs in the SELinux framework. It also does not allow any more access than Permissive mode.

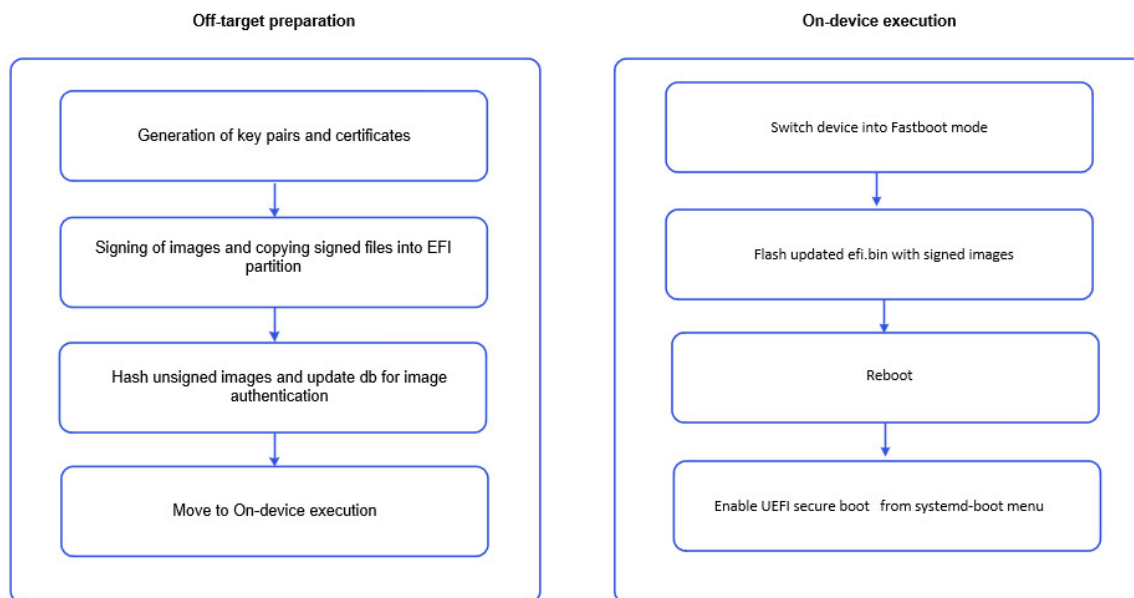
To disable the feature for testing, remove SELinux from `DISTRO_FEATURES`:

```
conf/distro/include/qcom-base.inc
-- DISTRO_FEATURES:append = " selinux"
```

9.3 Enable UEFI secure boot

You can setup an initial UEFI secure boot configuration, and convert keys and certificates into a format that UEFI can understand.

See the workflow to understand the off-target preparation and the on-device execution.



Prerequisites to generate UEFI secure boot key

You should ensure to install the following tools:

Tool	Description
OpenSSL 0.9.80 June 2010 (or later version)	This tool must be installed on the Linux host machine.
Cert-to-efi-sig-list	This tool converts OpenSSL certificates to EFI signature lists. It is installed as part of the <code>efitools</code> package.
Sign-efi-sig-list	This tool signs the EFI signature list, which is essential for managing secure variables in UEFI. It is installed as part of the <code>efitools</code> package.
Hash-efi-sig-list	This tool creates a hash signature list entry from a binary.

9.3.1 Generate key and certificate

To enable UEFI secure boot, generate a pair of keys and certificates for signing and authentication.

The key generation supports the following algorithms:

- RSA 2048/4096 with SHA-256/SHA384 hash algorithm
- ECDSA secp256r1/secp384r1

The following procedures provide instructions to generate keys and certificates with RSA 2048 and SHA-256 as an example.

NOTE Create a folder and execute the commands in the same location and ensure to perform these steps on a Linux machine.

NOTE For ECC, replace `rsa:2048` with `ec:secp384r1` or `ec:secp256r1`. For SHA384, replace `-sha256` with `-sha384` in the following commands.

Generate UID

You can generate a GUID and create three new keys with self-signed certificates in CRT/PEM format and keys in .key format:

GUID Use `uuidgen` to generate the signature owner GUID:

```
uuidgen --random > GUID.txt
```

Create PK key

1. Create a PK key (RSA-2048) pair and certificate:

```
openssl req -new -x509 -newkey rsa:2048 -subj "/CN=Custom PK/" -keyout  
PK.key -out PK.crt -days 3650 -nodes -sha256
```

2. Convert the .crt file into the .cer file:

```
openssl x509 -outform der -in PK.crt -out PK.cer
```

3. Convert the .crt file into the .esl file:

```
cert-to-efi-sig-list -g "$(< GUID.txt)" PK.crt PK.esl
```

4. Sign and generate the .auth file with the .crt, .esl, and .key files:

```
sign-efi-sig-list -k PK.key -c PK.crt PK PK.esl PK.auth
```

Create KEK key

1. Create a KEK key (RSA-2048) pair and certificate:

```
openssl req -new -x509 -newkey rsa:2048 -subj "/CN=Custom KEK/" -keyout  
KEK.key -out KEK.crt -days 3650 -nodes -sha256
```

2. Convert the .crt file into the .cer file:

```
openssl x509 -outform der -in KEK.crt -out KEK.cer
```

3. Convert the .crt file into the .esl file:

```
cert-to-efi-sig-list -g "$(< GUID.txt)" KEK.crt KEK.esl
```

4. Sign and generate the .auth file with the .crt, .esl, and .key files:

```
sign-efi-sig-list -k PK.key -c PK.crt KEK KEK.esl KEK.auth
```

Create DB key

1. Create a DB key (RSA-2048) pair and certificate:

```
openssl req -new -x509 -newkey rsa:2048 -subj "/CN=Custom DB Signing Key  
1/" -keyout db.key -out db.crt -days 3650 -nodes -sha256
```

2. Convert the .crt file into the .cer file:

```
openssl x509 -outform der -in db.crt -out db.cer
```

3. Convert the .crt file into the .esl file:

```
cert-to-efi-sig-list -g "$(< GUID.txt)" db.crt db.esl
```

4. Sign and generate the `.auth` file with the `.crt`, `.esl`, and `.key` files:

```
sign-efi-sig-list -k KEK.key -c KEK.crt db db.esl db.auth
```

9.3.2 Sign images and copy (.Auth) key/signed files to EFI partition

The EFI system partition consists of EFI, loader, and dtb directories with information relevant to the EFI when using the systemd-boot.

The procedure provides instructions to:

- Sign various images
- Copy (`.auth`) key and signed files to EFI partition directory
- Signed and executable images such as the `bootaa64.efi` file (systemd-boot) are placed in the `\EFI\BOOT\` directory and the `uki.efi` file (Linux) image is placed in the `\EFI\Linux\`.

In addition to validating signed images, systemd-boot is used to enroll keys.

- UEFI secure boot keys are placed in a specific folder in `/keys` for key enrollment. The systemd-boot uses these keys and provisions them in the RPMB or UEFI variable store during UEFI boot time services.
- You can configure the wait time (in seconds) in the systemd-boot loader configuration. Kernel loading is delayed during the wait time, allowing you to review and select available options in the systemd-boot menu.

Device tree files are stored in the `/dtb` folder. These files are used by UEFI during runtime, and the device tree files are initialized. While signing, `.sig` files are created and placed in the same folder as these files are non-PE images.

Table 9-1 EFI partition layout

EFI system partition (efi.bin)		
/EFI	/Loader	/dtb
/Boot - bootaa64.efi	loader.conf	▪ Chipset DTB file. ▪ Signature file for each DTB file.
/Linux - uki.efi	/keys -/authkeys PK.auth KEK.auth db.auth	

Follow these steps to place the signed images and keys in an EFI partition on a Linux host machine:

1. Locate the `efi.bin` file path in the `contents.xml`, file to obtain the `efi.bin` file from the meta.
2. Mount the `efi.bin` file into the `<workspace>` directory and create an `efimountedbin` folder within the `<workspace>` directory.
3. Mount the `efi.bin` file:

```
sudo mount efi.bin efimountedbin
cd efimountedbin
```

4. Create an authkeys folder within the `<workspace>/efimountedbin/loader/keys` directory to enroll keys.

5. Select and copy the .auth files (PK.auth, KEK.auth, and db.auth) to the authkeys folder.

```
sudo cp <selected algo PK/KEK/DB auth files from the files location>
<workspace>/efimountedbin/loader/keys/authkeys/
```

6. Sign the bootaa64.efi, uki.efi and dtb image files with the keys and copy to the respective folders in the efimountedbin directory.

- a. Sign efi images:

The sbsign tool is designed for signing EFI boot images, such as bootaa64.efi or UKI.efi, that follow EFI specifications. This tool, which is specifically used for UEFI secure boot signing, is available for download and use on Linux systems. It is important to note that sbsign can only sign PE images with an .efi extension.

- i. Copy the bootaa64.efi file from the efimountedbin folder/EFI/BOOT and the uki.efi file from the/EFI/Linux folder to the images folder on your Linux machine.

- ii. Sign the images:

```
cd <workspace>/images
```

```
sudo sbsign --key <workspace>/keys/db.key --cert <workspace>/keys/
db.crt bootaa64.efi --output <workspace>/bootaa64.efi
```

```
sudo sbsign --key <workspace>/keys/db.key --cert <workspace>/keys/
db.crt uki.efi --output <workspace>/uki.efi
```

- b. Sign dtb image:

All images authenticated by UEFI secure boot are regular APIs and typically in the PE format. The signature header and size are appended to the existing PE header, and signature is appended at the end of the signed file.

However, when images in non-PE formats require UEFI secure boot authentication, the absence of the PE header and its magic number to recognize the image format fail. As a result, it is not possible to use standard tools and paths for image verification.

Currently, among the list of images that UEFI secure boot verifies, only the dtb files are in non-PE format images. As an alternative to sbsign tool, you can use the OpenSSL cms command to generate signature files for signing images in non-PE format.

Follow these steps for signing non-EFI images:

- i. To sign the dtb file and signature file, run the following command:

```
openssl cms -sign -inkey <.key file> -signer <.crt file> -binary
-in <input dtb file>--out <Output .dtb.sig file> -outform DER
```

- ii. To sign the image, run the following command:

```
cd <workspace>/images
```

```
sudo openssl cms -sign -inkey <workspace>/keys/db.key -signer
<workspace>/keys/db.crt -binary -in qcm6490-idp.dtb --out qcm6490-
idp.sig -outform DER
```

7. Copy the signed uki and bootaa64 images back to their respective folder locations (/EFI/BOOT and /EFI/Linux) within the efimountedbin directory.

8. Copy the signed `qcm6490-idp.sig` file to the `/dtb` folder of the `efimountedbin` directory.
9. Configure the wait time in `systemd-boot`:
 - a. Open and edit the `loader.conf` file at `/loader/loader.conf` with `sudo` access:


```
sudo vi loader.conf
```
 - b. Add the line `timeout 2` to set the boot menu timeout and save the file.
10. To unmount the EFI binary to retrieve the latest `efi.bin` file, execute the command:


```
sudo umount efimountedbin
```
11. Securely place the signed images and keys in the EFI partition on target.
 Bring the device into the Fastboot mode and flash the latest `efi.bin` file with the `fastboot` command:


```
fastboot flash efi <efi binary location>
```

9.3.3 Enable UEFI secure boot from systemd-boot menu

The EFI binary is composed of signed images and secure boot keys, which are generated and then flashed into the system. For more details, see [Sign images and copy \(.Auth\) key/signed files to EFI partition](#).

When the UEFI is loaded and executed during the next bootup, the `systemd-boot` manager displays the **EnrollSecure Boot keys: authkeys** and **Ubuntu1 8.04.6LTS** menu options on the screen

NOTE These options are displayed when a timeout is configured. For more information, see `loader.conf` settings in [Sign images and copy \(.Auth\) key/signed files to EFI partition](#).



Figure 9-2 Systemd-boot menu options

You can use the volume `+/-` buttons to navigate and select the appropriate option to enroll the keys.

When the key is successfully enrolled, UEFI automatically switches from **SetupMode** to **UserMode**. The logs for the UEFI secure boot enablement are listed in the serial logs when the `systemd-boot` triggers a system reset to apply the changes.

By default, UEFI starts in **UserMode**, and the UEFI secure boot is initialized during the subsequent boot cycle. The logs for the UEFI secure boot enablement are listed in the serial logs when the `systemd-boot` transfers control to the `KERNEL EFI STUB`.

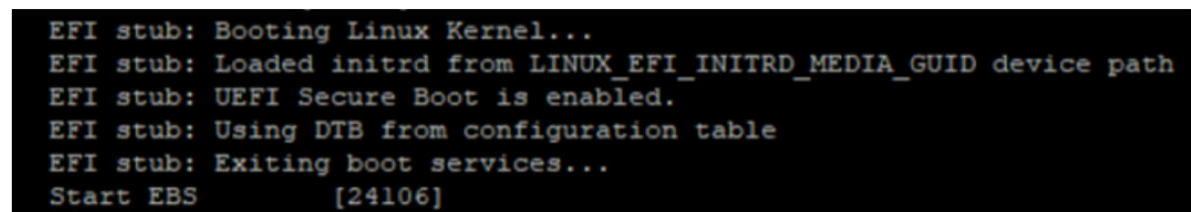


Figure 9-3 UEFI secure boot enablement information from serial log

The option to enroll with systemd-boot is only available once. This release does not support the re-provisioning and updating of UEFI secure boot keys.

9.3.4 Hash unsigned images and update db for image authentication

UEFI secure boot allows image authentication. This authentication is achieved through the hash of images stored in the signature database (DB), even if the images are not signed or the certificates in the images are not present in the DB.

This process is reserved for content that cannot be signed or altered from its vendor-provided state. If the image hash is available in the DBX deny list, the trust of signed binaries can be removed without having to revoke the corresponding certificates or keys. This is useful, for example, when dealing with a previously signed boot loader that is vulnerable to recent exploits.

It is redundant to apply a signature and create a DB hash for the same binary. Follow these steps if the image composition does not require any changes, meaning no new keys and certificates are being added or modified in the image, and no UEFI secure boot authentication is needed for the existing images.

Generate db.auth file for unsigned images

You can calculate the hash of images and generate an allowed signature DB file by following these steps:

1. Generate a hash of all images to be verified and convert the hash into an .esl file:

```
hash-to-efi-sig-list <list of efis to be hashed> <output file name  
with .esl extension>
```

2. Sign the hash .esl file with the DB key:

```
sign-efi-sig-list -k <.key file location> -c <.crt file location>  
<secure variable name> <Above generated .esl file> <o/p .auth file>
```

3. Copy the generated db.auth file into the EFI binary and provision the keys into the device.

For example, on a Linux host machine:

1. Mount the efi.bin file to the <workspace> directory and create an efimountedbin folder in the <workspace> directory.
2. Create a testkeys folder in the <workspace> directory on the Linux machine and copy the pre-existing keys to it.

3. Sign the images:

```
hash-to-efi-sig-list <workspace>/efimountedbin/EFI/BOOT/ bootaa64.efi  
<workspace>/efimountedbin/EFI/Linux/uki.efi mergedhash.esl  
sign-efi-sig-list -k keys db.key -c db.crt db mergedhash.esl db.auth
```

4. Copy the db.auth file to the qckeys folder at <workspace>/efimountedbin/loader/keys/qckeys.

5. Follow the dtb signing steps and sign the dtb images to generate a new `efi.bin` file. For more information, see [Sign images and copy \(.Auth\) key/signed files to EFI partition](#).
6. For a Linux host machine on target:
 - a. Erase any existing UEFI secure boot keys and flash the EFI binary with fastboot.
 - b. Provision keys with systemd-boot. For more information, see [Enable UEFI secure boot from systemd-boot menu](#).

NOTE All unsigned files are signed with other keys and authenticated with UEFI using this method.

9.4 Sample OpenSSL configuration

The sample shows the configuration file used to generate keys and certificate requests..

```
#
# Copyright (c) 2013 Qualcomm Technologies, Inc.
# All Rights Reserved.
# Confidential and Proprietary - Qualcomm Technologies, Inc.
#
# OpenSSL example configuration file.
# This is mostly being used for generation of certificate requests.
#

# This definition stops the following lines choking if HOME isn't
# defined.
HOME            = .
RANDFILE        = $ENV::HOME/.rnd

# Extra OBJECT IDENTIFIER info:
#oid_file        = $ENV::HOME/.oid
oid_section      = new_oids

# To use this configuration file with the "-extfile" option of the
# "openssl x509" utility, name here the section containing the
# X.509v3 extensions to use:
# extensions      =
# (Alternatively, use a configuration file that has only
# X.509v3 extensions in its main [= default] section.)

[ new_oids ]

# We can add new OIDs in here for use by 'ca' and 'req'.
# Add a simple OID like this:
# testoid1=1.2.3.4
# Or use config file substitution like this:
# testoid2=${testoid1}.5.6
```

```
#####
[ ca ]
default_ca    = CA_default          # The default ca section

#####
[ CA_default ]

dir           = ./demoCA            # Where everything is kept
certs         = $dir/certs          # Where the issued certs are kept
crl_dir       = $dir/crl            # Where the issued crl are kept
database      = $dir/index.txt      # database index file.
#unique_subject = no                # Set to 'no' to allow creation of
                                    # several certificates with same subject.
new_certs_dir = $dir/newcerts       # default place for new certs.

certificate   = $dir/cacert.pem     # The CA certificate
serial        = $dir/serial         # The current serial number
crlnumber     = $dir/crlnumber      # the current crl number
                                    # must be commented out to leave a V1 CRL
crl           = $dir/crl.pem        # The current CRL
private_key   = $dir/private/cakey.pem # The private key
RANDFILE      = $dir/private/.rand  # private random number file

x509_extensions = usr_cert         # The extensions to add to the cert

# Comment out the following two lines for the "traditional"
# (and highly broken) format.
name_opt      = ca_default          # Subject Name options
cert_opt      = ca_default          # Certificate field options

# Extension copying option: use with caution.
# copy_extensions = copy

# Extensions to add to a CRL. Note: Netscape communicator chokes on V2 CRLs
# so this is commented out by default to leave a V1 CRL.
# crlnumber must also be commented out to leave a V1 CRL.
# crl_extensions = crl_ext

default_days   = 365                # how long to certify for
default_crl_days = 30               # how long before next CRL
default_md     = sha1               # which md to use.
preserve      = no                  # keep passed DN ordering

# A few difference way of specifying how similar the request should look
# For type CA, the listed attributes must be the same, and the optional
# and supplied fields are just that :-)
policy        = policy_match
```

```

# For the CA policy
[ policy_match ]
countryName          = match
stateOrProvinceName  = match
organizationName     = match
organizationalUnitName = optional
commonName           = supplied
emailAddress         = optional

# For the 'anything' policy
# At this point in time, you must list all acceptable 'object'
# types.
[ policy_anything ]
countryName          = optional
stateOrProvinceName  = optional
localityName         = optional
organizationName     = optional
organizationalUnitName = optional
commonName           = supplied
emailAddress         = optional

#####
[ req ]
default_bits          = 1024
default_keyfile        = privkey.pem
distinguished_name     = req_distinguished_name
attributes            = req_attributes
x509_extensions       = v3_ca      # The extensions to add to the self signed cert

# Passwords for private keys if not present they will be prompted for
# input_password = secret
# output_password = secret

# This sets a mask for permitted string types. There are several options.
# default: PrintableString, T61String, BMPString.
# pkix      : PrintableString, BMPString.
# utf8only: only UTF8Strings.
# nombstr : PrintableString, T61String (no BMPStrings or UTF8Strings).
# MASK:XXXX a literal mask value.
# WARNING: current versions of Netscape crash on BMPStrings or UTF8Strings
# so use this option with caution!
string_mask = nombstr

req_extensions = v3_req # The extensions to add to a certificate request

[ req_distinguished_name ]

```

```
countryName          = Country Name (2 letter code)
countryName_default  = AU
countryName_min      = 2
countryName_max      = 2

stateOrProvinceName  = State or Province Name (full name)
stateOrProvinceName_default = Some-State

localityName         = Locality Name (eg, city)

0.organizationName    = Organization Name (eg, company)
0.organizationName_default = Internet Widgits Pty Ltd

# we can do this but it is not needed normally :-)
#1.organizationName    = Second Organization Name (eg, company)
#1.organizationName_default = World Wide Web Pty Ltd

organizationalUnitName = Organizational Unit Name (eg, section)
#organizationalUnitName_default =

commonName            = Common Name (eg, YOUR name)
commonName_max        = 64

emailAddress          = Email Address
emailAddress_max       = 64

# SET-ex3             = SET extension number 3

[ req_attributes ]
challengePassword      = A challenge password
challengePassword_min  = 4
challengePassword_max  = 20

unstructuredName       = An optional company name

[ usr_cert ]

# These extensions are added when 'ca' signs a request.

# This goes against PKIX guidelines but some CAs do it and some software
# requires this to avoid interpreting an end user certificate as a CA.

basicConstraints=CA:FALSE
keyUsage = nonRepudiation, digitalSignature, keyEncipherment

# Here are some examples of the usage of nsCertType. If it is omitted
# the certificate can be used for anything *except* object signing.
```

```
# This is OK for an SSL server.
# nsCertType = server

# For an object signing certificate this would be used.
# nsCertType = objsign

# For normal client use this is typical
# nsCertType = client, email

# and for everything including object signing:
# nsCertType = client, email, objsign

# This is typical in keyUsage for a client certificate.
# keyUsage = nonRepudiation, digitalSignature, keyEncipherment

# This will be displayed in Netscape's comment listbox.
nsComment = "OpenSSL Generated Certificate"

# PKIX recommendations harmless if included in all certificates.
subjectKeyIdentifier=hash
authorityKeyIdentifier=keyid,issuer

# This stuff is for subjectAltName and issuerAltnames.
# Import the email address.
# subjectAltName=email:copy
# An alternative to produce certificates that aren't
# deprecated according to PKIX.
# subjectAltName=email:move

# Copy subject details
# issuerAltName=issuer:copy

#nsCaRevocationUrl = http://www.domain.dom/ca-crl.pem
#nsBaseUrl
#nsRevocationUrl
#nsRenewalUrl
#nsCaPolicyUrl
#nsSslServerName

[ v3_req ]

# Extensions to add to a certificate request

subjectKeyIdentifier=hash

#authorityKeyIdentifier=keyid:always,issuer:always
```

```
basicConstraints = CA:FALSE
keyUsage = nonRepudiation, digitalSignature, keyEncipherment

[ v3_ca ]

# Extensions for a typical CA

# PKIX recommendation.

subjectKeyIdentifier=hash

#authorityKeyIdentifier=keyid:always,issuer:always

# This is what PKIX recommends but some broken software chokes on critical
# extensions.
#basicConstraints = critical,CA:true
# So we do this instead.
basicConstraints = CA:true

# Key usage: this is typical for a CA certificate. However since it will
# prevent it being used as an test self-signed certificate it is best
# left out by default.
keyUsage = cRLSign, keyCertSign

# Some might want this also
# nsCertType = sslCA, emailCA

# Include email address in subject alt name: another PKIX recommendation
# subjectAltName=email:copy
# Copy issuer details
# issuerAltName=issuer:copy

# DER hex encoding of an extension: beware experts only!
# obj=DER:02:03
# Where 'obj' is a standard or added object
# You can even override a supported extension:
# basicConstraints= critical, DER:30:03:01:01:FF

[ crl_ext ]

# CRL extensions.
# Only issuerAltName and authorityKeyIdentifier make any sense in a CRL.

# issuerAltName=issuer:copy
```

```
authorityKeyIdentifier=keyid:always,issuer:always

[ proxy_cert_ext ]
# These extensions should be added when creating a proxy certificate

# This goes against PKIX guidelines but some CAs do it and some software
# requires this to avoid interpreting an end user certificate as a CA.

basicConstraints=CA:FALSE

# Here are some examples of the usage of nsCertType. If it is omitted
# the certificate can be used for anything *except* object signing.

# This is OK for an SSL server.
# nsCertType          = server

# For an object signing certificate this would be used.
# nsCertType = objsign

# For normal client use this is typical
# nsCertType = client, email

# and for everything including object signing:
# nsCertType = client, email, objsign

# This is typical in keyUsage for a client certificate.
# keyUsage = nonRepudiation, digitalSignature, keyEncipherment

# This will be displayed in Netscape's comment listbox.
nsComment          = "OpenSSL Generated Certificate"

# PKIX recommendations harmless if included in all certificates.
subjectKeyIdentifier=hash
authorityKeyIdentifier=keyid,issuer:always

# This stuff is for subjectAltName and issuerAltname.
# Import the email address.
# subjectAltName=email:copy
# An alternative to produce certificates that aren't
# deprecated according to PKIX.
# subjectAltName=email:move

# Copy subject details
# issuerAltName=issuer:copy

#nsCaRevocationUrl      = http://www.domain.dom/ca-crl.pem
#nsBaseUrl
```

```
#nsRevocationUrl
#nsRenewalUrl
#nsCaPolicyUrl
#nsSslServerName

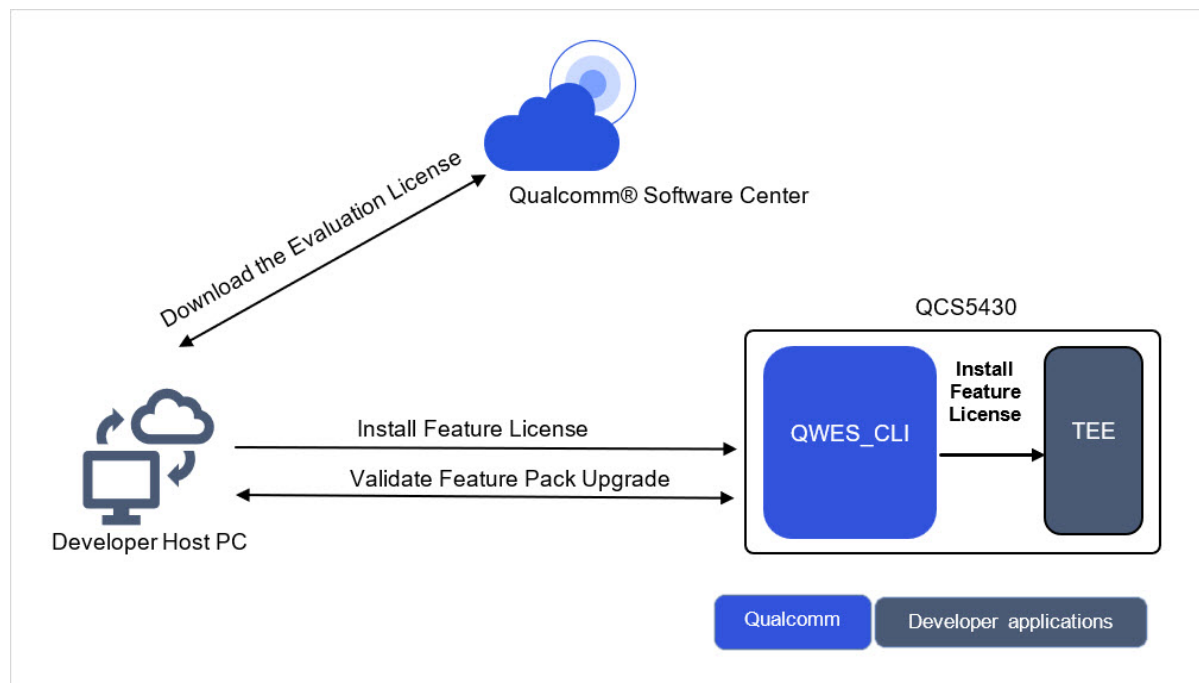
# This really needs to be in place for it to be a proxy certificate.
proxyCertInfo=critical,language:id-ppl-anyLanguage,pathlen:3,policy:foo
```

9.5 Install or upgrade QCS5430 SoftSKU feature packs

You can upgrade the QCS5430 soft stock keeping unit (SKU) feature packs using the Qualcomm® wireless edge services (Qualcomm WES) license.

For more information on feature packs, see [QCS5430: Feature packs](#).

The figure shows the process of upgrading the QCS5430 SoftSKU feature packs:



Download evaluation license

Download the evaluation license according to the feature packs listed in the table:

Feature packs	Download links to license files
FP2	QCM5430 FP2.0
FP2.5	This information will be updated in a future release.
FP3	This information will be updated in a future release.

Prepare device

1. Set up the Wi-Fi on the device and get the device-ip-address.

2. Connect to the device via SSH:

```
ssh root@device-ip-address
```

For instructions, see [Qualcomm Linux Build Guide → How to → How to SSH](#).

3. Push the license to the device:

```
scp /path/to/<QCM5430 FP*>.pfm root@device-ip-address:/data/
```

Install evaluation license

1. Ensure you have the following prerequisites:

- RPMB is provisioned.
- `qwes_cli` tool is available in the device.

2. Install the Qualcomm WES command-line interface tools:

- Help: `qwes_cli -help`.
- Default tools shipped with the Qualcomm Linux software product.

3. Install the evaluation license:

```
ssh root@device-ip-address
qwes_cli -f /temp/<QCM5430 FP*>.pfm install
```

4. Verify license installation:

```
qwes_cli list
```

Command	Output
Check the serial number: <code>qwes_cli list</code>	<pre># quwes_cli list Running GetAllSerialNumber test with multithread Disabled Success. Len = 25 Parsing 2 serial numbers Serial Number: 15b3 Serial Number: 3e52b512f2f47851a40fa5b30000018c86261aa1</pre>

Ensure you reboot the device for the license to take effect.

Verify feature pack upgrade

There are two ways to verify the feature pack:

- Verify the feature pack upgrade by using the following commands:

```
ssh root@device-ip-address
cat /proc/device-tree/model
```

The feature pack ID model is displayed in the output:

Qualcomm Technologies, Inc. qcs5430 **fp2** addons rb3gen2 platforms.

NOTE The feature pack and model ID mapping will be {Feature Pack2: fp2 ; Feature pack2.5 : fp2.5; Feature pack 3: fp3}.

- Verify feature pack ID from the following UEFI logs:

FLP1	SoftSKU ID is disabled. The default setting is 1.
FP2	SoftSKU ID enabled. updated here: 2

Manage device licenses

You can perform the following operations to manage your device licenses:

- Upgrade the feature pack or apply a new license.
 - Multiple licenses can be installed.
 - License with the highest feature pack will be enforced by design.
 - You can upgrade or downgrade the feature pack license among the offered.
 - To downgrade, remove the license with the higher feature pack and apply the license for the required feature pack.
- Remove the evaluation license.
 - Get the license serial number:


```
qwes_cli list
Running GetAllSerialNumber test with multithread Disabled
Success. Len = 25
Parsing 2 serial numbers
Serial Number: 15b3
Serial Number: 3e52b512f2f47851a40fa5b30000018c86261aa1
```
 - Remove the license:


```
ssh root@device-ip-address
qwes_cli -s 3e52b512f2f47851a40fa5b30000018c86261aa1 remove
Serial number length 20
Remove License is successful.
```
 - Verify if the removed license serial number does not appear in the license list:


```
qwes_cli list
Running GetAllSerialNumber test with multithread Disabled
Success. Len = 4
Parsing 1 serial numbers
Serial Number: 15b3
```

Feature pack license – Persistence over software upgrades

- The license is installed in persistent secure storage, RPMB.
- When the license is installed successfully, its metadata is stored in RPMB, and a copy is maintained in the data partition.
- Even if a user data is erased, the license is already preserved in RPMB, and the device functionality is not impacted.

NOTE Users who have full access to the proprietary software shipped with Qualcomm Linux can manage feature pack licenses. If you have access, see [Qualcomm Linux Wireless Edge Services Guide](#).

10 Customize

To enhance the TA memory, you need to customize the memory, and build and customize the SEPolicy.

Memory customization

If you have full access to the proprietary software shipped with Qualcomm Linux, you can customize the memory. If you have access, see [Qualcomm Linux Security Guide - Addendum → Memory customization](#).

SEPolicy customization

Qualcomm SEPolicy is dependent on the upstream SEPolicy. Therefore, the upstream SEPolicy's make system is used for building and customizing the SEPolicy.

Any customization to upstream code must be stored in the [patches](#) folder as a path to the upstream code. You can find the upstream code at <https://github.com/SELinuxProject/refpolicy/>

The Qualcomm code is configured to the monolithic SEPolicy mode (a single SEPolicy image).

To compile the SEPolicy, run the following commands:

```
export SHELL=/bin/bash
MACHINE=qcm6490 DISTRO=qcom-wayland source setup-environment
bitbake refpolicy-mls.
```

You can also modify and build incrementally.

The audit2allow and research tools on Ubuntu 18 or 20 do not support policy version 33. The policy version33 is supported from Ubuntu 23. If you are not using Ubuntu 23, you can use a docker setup or a virtual machine to run audit2allow.

The following command is used to install docker on Ubuntu 23.04. If the command prompt does not change when `policy.33` is pulled from the `target /build`, and if `denials.txt` is a file capturing the denials, use the following command:

```
audit2allow -i denails.txt -p policy.33
```

If this command is not found, then use the following command to install the required package:

```
sudo apt install policycoreutils-python-utils
```

11 Debug

Debug provides a set of common logging and debugging techniques to troubleshoot issues in Qualcomm TEE, TA, CA, and secure devices.

NOTE Remember to run all the SSH commands in the SELinux Permissive mode. The Enforcing mode will be supported in the future. For instructions on how to connect to the device, see [Qualcomm Linux Build Guide](#) → [How to](#) → [How to SSH](#).

Debug Qualcomm TEE

Qualcomm TEE kernel logs, also known as the TZDiag log, can be used to debug errors that occur in Qualcomm TEE.

The TZDiag log is available in the Linux kernel driver, which redirects the logs.

1. Connect to the device as the root via SSH.
2. Capture the TZ logs using the following command:

```
cat /proc/tzdbg/log > tzbsp_log.txt
```

The error codes in `tzbsp_log.txt` are encoded in hexadecimal. You can run the following tool to decode `tzbsp_log.txt` from hexadecimal to string.

1. Navigate to `<TZ.XF.X.X path>/trustzone_images/ssg/bsp/tz/build/tz/A53_64/<BuildFlavor>`
2. Run the following commands using python 3.

```
python3 print_tz_log.py -l tzbsp_log.txt -e errorCodesDict.txt -t  
<location of TZ build path> -o tzbsp_log_decode.txt
```

For example:

```
Python3 print_tz_log.py -l tzbsp_log.txt -e errorCodesDict.txt -t \\crmhyd  
\nsid-hyd-05\TZ.XF.5.0-07927-KODIAKAAAAANAAZT-1 -o tzbsp_log_decode.txt
```

For device log collection, the TZ Diag log buffer is part of the RAM dump, which can be parsed using `qsee.elf` from TZ.XF software in the crash dump parser tool. For offline or off-device log collection, the TZ Diag log buffer is part of the RAM dump, which can be parsed using `qsee.elf` (`trustzone_images/ssg/bsp/qsee/build/${tz_bid:EACAANAA}`) from the TZ.XF software in the crash dump parser tool.

Debug using secure crash dump

You can debug QualcommTEE using the RAM dump. The execution region dump of Qualcomm TEE is collected via secure crash dumps.

Devices that trigger the fuse with stage 2 sec.elf are known as secure boot-enabled devices. To debug on these devices, see [Sectools v2: Secure Debug User Guide](#).

Debug TA and CA

TA logs, also known as Qualcomm TEE logs, are used to debug the errors in TAs. To debug errors in the CA, kernel and journalctl logs are used.

For online or on-device log collection, Linux collects the Qualcomm TEE/kernel logs at runtime. You can connect to the device via SSH and use the following commands:

- To collect the Qualcomm TEE logs from Linux:

```
cat /proc/tzdbg/qsee_log > qsee_log.txt
```

- For CA, to collect the kernel and logcat logs:

```
cat /dev/kmsg > kernel_log.txt
journalctl > journalctl.txt
```

- For offline or off-device log collection, the Qualcomm TEE log is available in RAM dumps along with the kernel and journalctl logs.

Debug on secure devices

As part of the secure boot procedure, blowing debug disable fuses will disable debugging capabilities on the devices. This includes RAM dumps, INV, and NINV debug on the subsystems.

The debug policy feature allows control over the debug capability for a device enabled with secure boot.

The debug policy image enables debug capabilities such as JTAG re-enable (INV debug), RAM dump, and TZ logging (NINV debug) on commercial secure devices.

For security reasons, the serial number of the device controls the debug policy for secure RAM dumps, Qualcomm TEE logs, and JTAG.

Enabling JTAG on the Qualcomm TEE subsystem disables the device's security with respect to hardware key generation. As a result, existing secure storage like user data, SFS, and RPMB becomes inaccessible. Sometimes, the device may prompt for a factory data reset. Use the following command to debug on secure devices:

```
<meta>/common/sectoolsv2/ext/<platform>/sectools.exe secure-debug --security-  
profile <meta>/common/sectoolsv2/<chipset>_security_profile.xml --generate --  
outfile apdp_out.mbn --all-flags --sign --signing-mode LOCAL --oem-id=0x1 --  
root-certificate=./RSA-OEM-KEYS/qpsa_rootca.cer --ca-certificate=./RSA-OEM-  
KEYS/qpsa_attestca.cer --ca-key=./RSA-OEM-KEYS/qpsa_attestca.key --oem-  
product-id=0xabcd --serial-number=0xabcdabcd
```

Ensure that you configure the OEM_ID, PRODUCT_ID, serial number and keys, and certification paths appropriately.

For more information, see [Sectools v2: Secure Debug User Guide](#).

Flash APDP on device

To flash APDP on the device, run the following command

```
Fastboot flash apdp <path to apdp.mbn>
```

Table 11-1 Debug policy flags for dump collection

Stage	Full dump			Mini dump
Stage	Applications (DCC and scan dump) ADSP/ Video/RPM/ SLPI	Modem/ Qualcomm TEE/Secure dump	TZDiag	–
Non-secure	No debug policy needed	No debug policy needed	No debug policy needed	No debug policy needed
Stages 1 secure	No APDP image needed			No APDP image needed
Stages 2 secure	--nonsecure -crash-dumps	--offline-crash-dumps with device serial number	--logs with device serial number Or Encrypted TZ diag with --nonsecure-crash-dumps + TZDiag encryption public key/exp in devcfg can be configured in the following location: \trustzone_images\ssg \securemsm\ trustzone\qsee\mink \oem\config<chipset> \oem_config.xml	▪ Apps minidump: --apps-encrypted-mini-dumps ▪ Modem and WLAN: ▫ --mpss-encrypted-mini-dumps ▫ --wlan-encrypted-mini-dumps ▪ aDSP minidump:--adsp-encrypted-mini-dumps ▪ cDSP minidump:--cdsp-encrypted-mini-dumps For more information, see MiniDump Software User Guide.

12 Examples

If you have full access to the proprietary software shipped with Qualcomm Linux, you can view the examples to load TA and CA. If you have access, see [Qualcomm Linux Security Guide - Addendum → Examples](#).

A References

Related documents

Title	Document number
MiniDump Software User Guide	80-P8754-71
Qualcomm Linux Build Guide	80-70014-254
Qualcomm Linux Kernel Guide	80-70014-3
Sectools v2: Secure Debug User Guide	80-NM248-23
SecTools V2: Metabuild Secure Image User Guide	80-NM248-17
SecTools V2: Fuse Blower User Guide	80-NM248-9
SecTools V2: ELF Tool User Guide	80-NM248-18
SecTools V2: MBN Tool User Guide	80-NM248-19
SecTools V2: ELF Consolidator User Guide	80-NM248-20
SecTools V2: Secure Image User Guide	80-NM248-12

Acronyms and terms

Acronym or term	Definition
CA	Client applications
DRM	Digital rights management
EL	Exception level
HAL	Hardware abstraction layer
HLOS	High-level operating system
HMAC	Hashed message authentication code
I2C	Inter-integrated circuit
ICE	Inline crypto engine
MAC	Message authentication code
MINK	Mini Kernel
MPU	Memory protection unit
OCIMEM	On-chip internal memory
OEM	Original equipment manufacturer
PIL	Peripheral image loader
pIMEM	Protected memory

Acronym or term	Definition
RMA	Returned material for analysis
Qualcomm TEE	Qualcomm Trusted Execution Environment
RPMB	Replay protected memory block
SFS	Secure file system
SKU	Stock keeping unit
SMC	Secure monitor call
SPI	Serial peripheral interface
SSL	Secure sockets layer
TA	Trusted application
TZ	TrustZone
TZBSP	TrustZone board support package
UFS	Universal flash storage
VM	Virtual machine
XBL	eXtensible boot loader
xPU	External protection unit

LEGAL INFORMATION

Your access to and use of this material, along with any documents, software, specifications, reference board files, drawings, diagnostics and other information contained herein (collectively this “Material”), is subject to your (including the corporation or other legal entity you represent, collectively “You” or “Your”) acceptance of the terms and conditions (“Terms of Use”) set forth below. If You do not agree to these Terms of Use, you may not use this Material and shall immediately destroy any copy thereof.

1) Legal Notice.

This Material is being made available to You solely for Your internal use with those products and service offerings of Qualcomm Technologies, Inc. (“Qualcomm Technologies”), its affiliates and/or licensors described in this Material, and shall not be used for any other purposes. If this Material is marked as “Qualcomm Internal Use Only”, no license is granted to You herein, and You must immediately (a) destroy or return this Material to Qualcomm Technologies, and (b) report Your receipt of this Material to qualcomm.support@qti.qualcomm.com. This Material may not be altered, edited, or modified in any way without Qualcomm Technologies’ prior written approval, nor may it be used for any machine learning or artificial intelligence development purpose which results, whether directly or indirectly, in the creation or development of an automated device, program, tool, algorithm, process, methodology, product and/or other output. Unauthorized use or disclosure of this Material or the information contained herein is strictly prohibited, and You agree to indemnify Qualcomm Technologies, its affiliates and licensors for any damages or losses suffered by Qualcomm Technologies, its affiliates and/or licensors for any such unauthorized uses or disclosures of this Material, in whole or part.

Qualcomm Technologies, its affiliates and/or licensors retain all rights and ownership in and to this Material. No license to any trademark, patent, copyright, mask work protection right or any other intellectual property right is either granted or implied by this Material or any information disclosed herein, including, but not limited to, any license to make, use, import or sell any product, service or technology offering embodying any of the information in this Material.

THIS MATERIAL IS BEING PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESSED, IMPLIED, STATUTORY OR OTHERWISE. TO THE MAXIMUM EXTENT PERMITTED BY LAW, QUALCOMM TECHNOLOGIES, ITS AFFILIATES AND/OR LICENSORS SPECIFICALLY DISCLAIM ALL WARRANTIES OF TITLE, MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, SATISFACTORY QUALITY, COMPLETENESS OR ACCURACY, AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MOREOVER, NEITHER QUALCOMM TECHNOLOGIES, NOR ANY OF ITS AFFILIATES AND/OR LICENSORS, SHALL BE LIABLE TO YOU OR ANY THIRD PARTY FOR ANY EXPENSES, LOSSES, USE, OR ACTIONS HOWSOEVER INCURRED OR UNDERTAKEN BY YOU IN RELIANCE ON THIS MATERIAL.

Certain product kits, tools and other items referenced in this Material may require You to accept additional terms and conditions before accessing or using those items.

Technical data specified in this Material may be subject to U.S. and other applicable export control laws. Transmission contrary to U.S. and any other applicable law is strictly prohibited.

Nothing in this Material is an offer to sell any of the components or devices referenced herein.

This Material is subject to change without further notification.

In the event of a conflict between these Terms of Use and the *Website Terms of Use* on www.qualcomm.com, the *Qualcomm Privacy Policy* referenced on www.qualcomm.com, or other legal statements or notices found on prior pages of the Material, these Terms of Use will control. In the event of a conflict between these Terms of Use and any other agreement (written or click-through, including, without limitation any non-disclosure agreement) executed by You and Qualcomm Technologies or a Qualcomm Technologies affiliate and/or licensor with respect to Your access to and use of this Material, the other agreement will control.

These Terms of Use shall be governed by and construed and enforced in accordance with the laws of the State of California, excluding the U.N. Convention on International Sale of Goods, without regard to conflict of laws principles. Any dispute, claim or controversy arising out of or relating to these Terms of Use, or the breach or validity hereof, shall be adjudicated only by a court of competent jurisdiction in the county of San Diego, State of California, and You hereby consent to the personal jurisdiction of such courts for that purpose.

2) Trademark and Product Attribution Statements.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Arm is a registered trademark of Arm Limited (or its subsidiaries) in the U.S. and/or elsewhere. The Bluetooth® word mark is a registered trademark owned by Bluetooth SIG, Inc. Other product and brand names referenced in this Material may be trademarks or registered trademarks of their respective owners.

Snapdragon and Qualcomm branded products referenced in this Material are products of Qualcomm Technologies, Inc. and/or its subsidiaries. Qualcomm patented technologies are licensed by Qualcomm Incorporated.